

Redis Tutorial



Redis tutorial provides basic and advanced concepts of Redis Database. Our Redis tutorial is designed for beginners and professionals both.

Redis is a No SQL database which works on the concept of key-value pair.

Our Redis Tutorial includes all topics of Redis such as what is redis, how to install redis, redis commands, data types, keys, lists, strings, hashes, sets, sorted sets, transaction, scripting, connection, server, interview questions etc.

What is Redis

Redis is a NoSQL database which follows the principle of key-value store. The key-value store provides ability to store some data called a value, inside a key. You can retrieve this data later only if you know the exact key used to store it.

Redis is a flexible, open-source (BSD licensed), in-memory data structure store, used as database, cache, and message broker. Redis is a NoSQL database so it facilitates users to store huge amount of data without the limit of a Relational database.

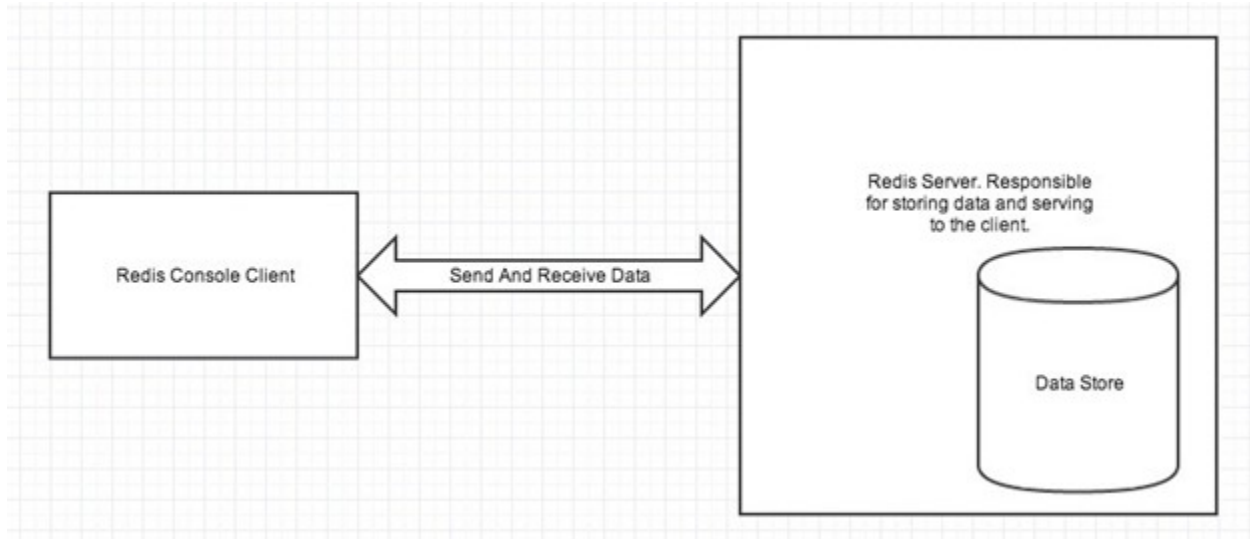
Redis supports various types of data structures like strings, hashes, lists, sets, sorted sets, bitmaps, hyperloglogs and geospatial indexes with radius queries.

Redis Architecture

There are two main processes in Redis architecture:

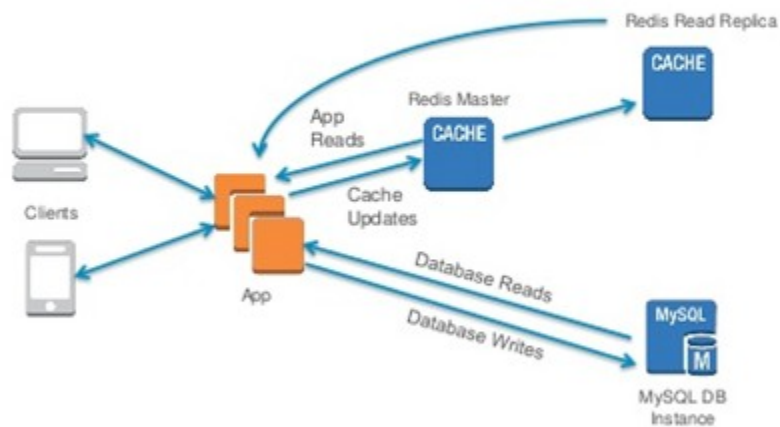
- Redis Client
- Redis Server

These client and server can be on same computer or two different computers.



Redis server is used to store data in memory . It controls all type of management and forms the main part of the architecture. You can create a Redis client or Redis console client when you install Redis application or you can use

Redis: Architecture



Redis Index

Redis Tutorial

- [Redis Tutorial](#)
- [Features of Redis](#)
- [Redis vs RDBMS](#)
- [Redis vs Other Key-value](#)
- [Installation on Windows](#)
- [Installation on Ubuntu](#)
- [Redis Configuration](#)
- [Redis Data types](#)
- [Redis All Commands](#)

Redis Commands

- [Redis Commands](#)
- [Redis Keys](#)
- [Redis Strings](#)
- [Redis Hashes](#)
- [Redis Lists](#)
- [Redis Sets](#)
- [Redis Sorted Sets](#)
- [Redis Transaction](#)
- [Redis Scripting](#)
- [Redis Connection](#)
- [Redis Server](#)

Redis Advance

- [Redis Backup & Restore](#)
- [Redis Security](#)
- [Redis Benchmarks](#)
- [Redis Client Connection](#)
- [Redis Pipelining](#)

- [Redis Partitioning](#)

Redis Differences

- [Redis vs Memcached](#)
- [Redis vs MongoDB](#)
- [Redis vs Elasticsearch](#)
- [Redis vs Aerospike](#)

Interview Questions

- [Redis Interview Questions](#)
-

Prerequisite

Before learning Redis, you must have the basic knowledge of SQL.

Audience

Our Redis Tutorial is designed to help beginners and professionals.

Problem

We assure that you will not find any problem in this Redis tutorial. But if there is any mistake, please post the problem in contact form.

Next Topic [Features of Redis](#)

Features of Redis

Following is the list of main features of Redis:

Speed: Redis stores the whole dataset in primary memory that's why it is extremely fast. It loads up to 110,000 SETs/second and 81,000 GETs/second can be retrieved in an entry level Linux box. Redis supports Pipelining of commands and facilitates you to use multiple values in a single command to speed up communication with the client libraries.

Persistence: While all the data lives in memory, changes are asynchronously saved on disk using flexible policies based on elapsed time and/or number of updates since last save. Redis supports an append-only file persistence mode. Check more on Persistence, or read the [AppendOnlyFileHowto](#) for more information.

Data Structures: Redis supports various types of data structures such as strings, hashes, sets, lists, sorted sets with range queries, bitmaps, hyperloglogs and geospatial indexes with radius queries.

Atomic Operations: Redis operations working on the different Data Types are atomic, so it is safe to set or increase a key, add and remove elements from a set, increase a counter etc.

Supported Languages: Redis supports a lot of languages such as ActionScript, C, C++, C#, Clojure, Common Lisp, D, Dart, Erlang, Go, Haskell, Haxe, Io, Java, JavaScript (Node.js), Julia, Lua, Objective-C, Perl, PHP, Pure Data, Python, R, Racket, Ruby, Rust, Scala, Smalltalk and Tcl.

Master/Slave Replication: Redis follows a very simple and fast Master/Slave replication. It takes only one line in the configuration file to set it up, and 21 seconds for a Slave to complete the initial sync of 10 MM key set on an Amazon EC2 instance.

Sharding: Redis supports sharding. It is very easy to distribute the dataset across multiple Redis instances, like other key-value store.

Portable: Redis is written in ANSI C and works in most POSIX systems like Linux, BSD, Mac OS X, Solaris, and so on. Redis is reported to compile and work under WIN32 if compiled with Cygwin, but there is no official support for Windows currently.

Next Topic [Redis vs RDBMS](#)

Redis vs RDBMS

Following is a list of differences between Redis and RDBMS:

Redis	RDBMS
Redis stores everything in primary memory.	RDBMS stores everything in secondary memory.
In Redis, Read and Write operations are extremely fast because of storing data in primary memory.	In RDBMS, Read and Write operations are slower because of storing data in secondary memory.
Primary memory is lesser in size and much expensive than secondary so, Redis cannot store large files or binary data.	Secondary memory is abundant in size compared to primary memory so, RDBMS can easily store large type of files.
Redis is used only to store those small textual information which needs to be accessed, modified and inserted at a very fast rate. If you try to write bulk data more than the available memory then you will receive errors.	RDBMS can hold large data which has less frequent access and is not required to be very fast.

Next Topic [Redis vs Other Key-value stores](#)

Redis vs Other Key-Value Stores

Key-value store is a special type of database storage system where data is stored in form of key and value pairs.

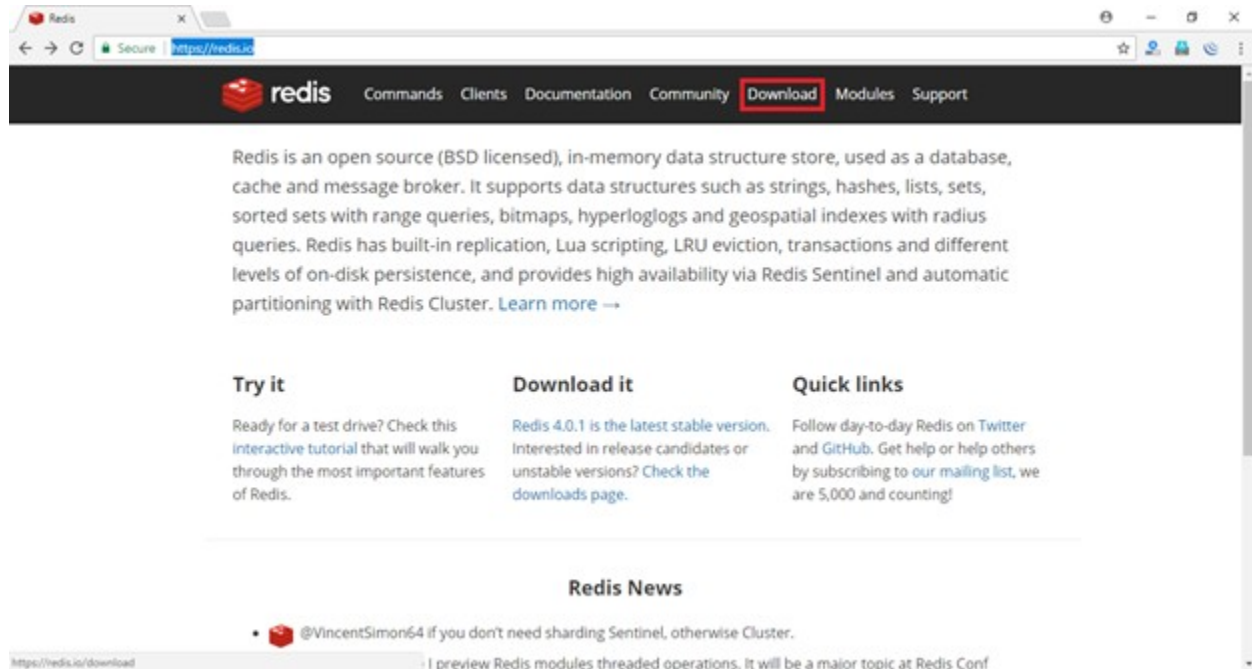
Redis is different compared to other key-value stores because of the following:

- Redis is a different evolution path in the key-value databases where values can contain more complex data types, with atomic operations defined on those data types.
- Redis data types are closely related to fundamental data structures and are exposed to the programmer as such, without additional abstraction layers.
- Redis is an in-memory but persistent on disk database, so it represents a different trade off where very high write and read speed is achieved with the limitation of data sets that can't be larger than memory.
- Another advantage of in memory databases is that the memory representation of complex data structures is much simpler to manipulate compared to the same data structure on disk, so Redis can do a lot, with little internal complexity.
- At the same time the two on-disk storage formats (RDB and AOF) don't need to be suitable for random access, so they are compact and always generated in an append-only fashion.

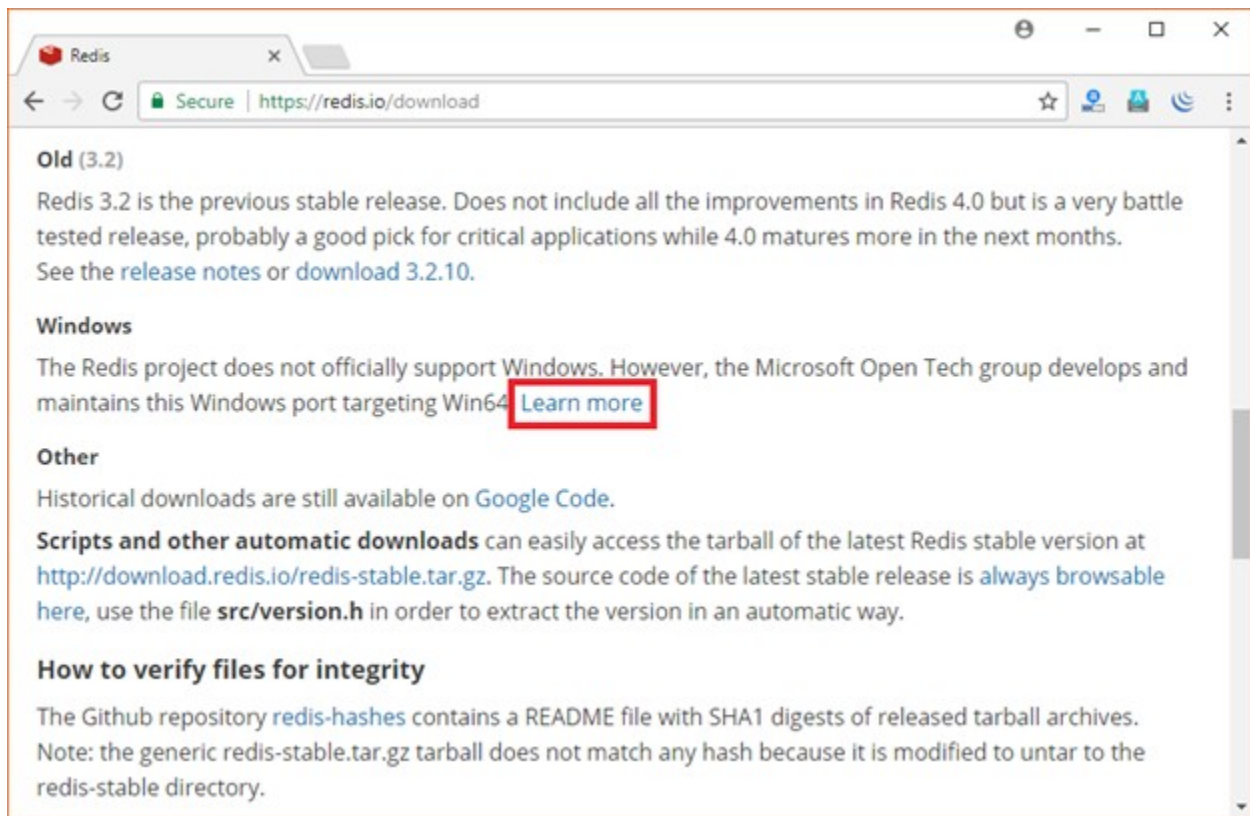
Next Topic [Redis Installation](#)

Redis Installation

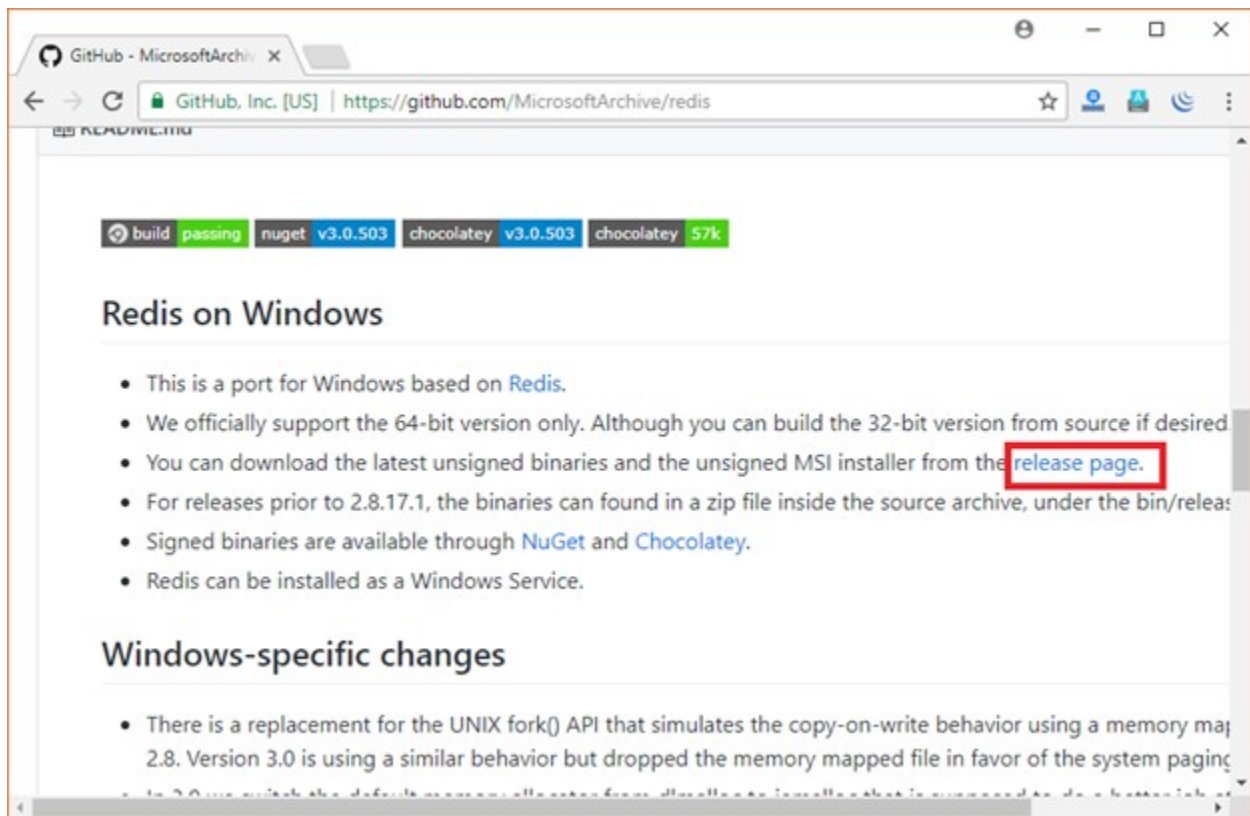
Go to Redis official website <https://redis.io/> and follow images given below:



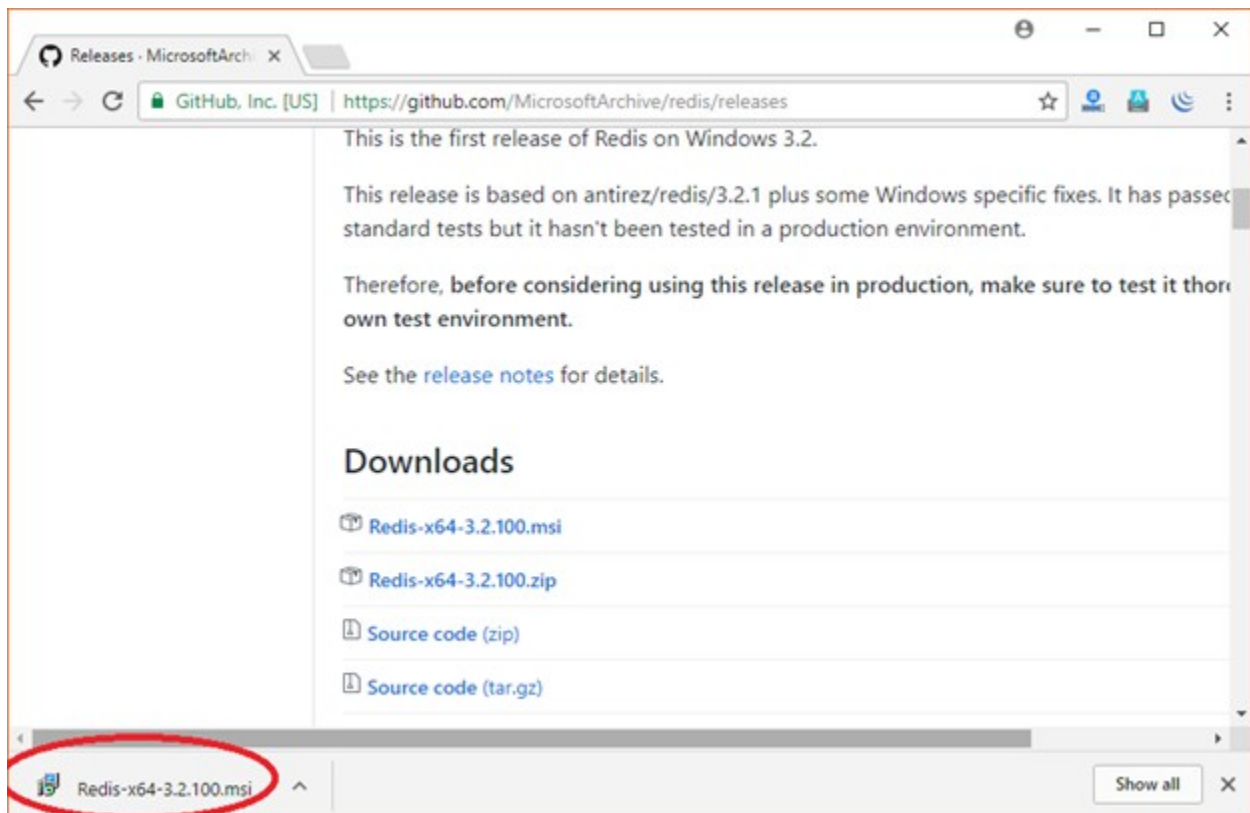
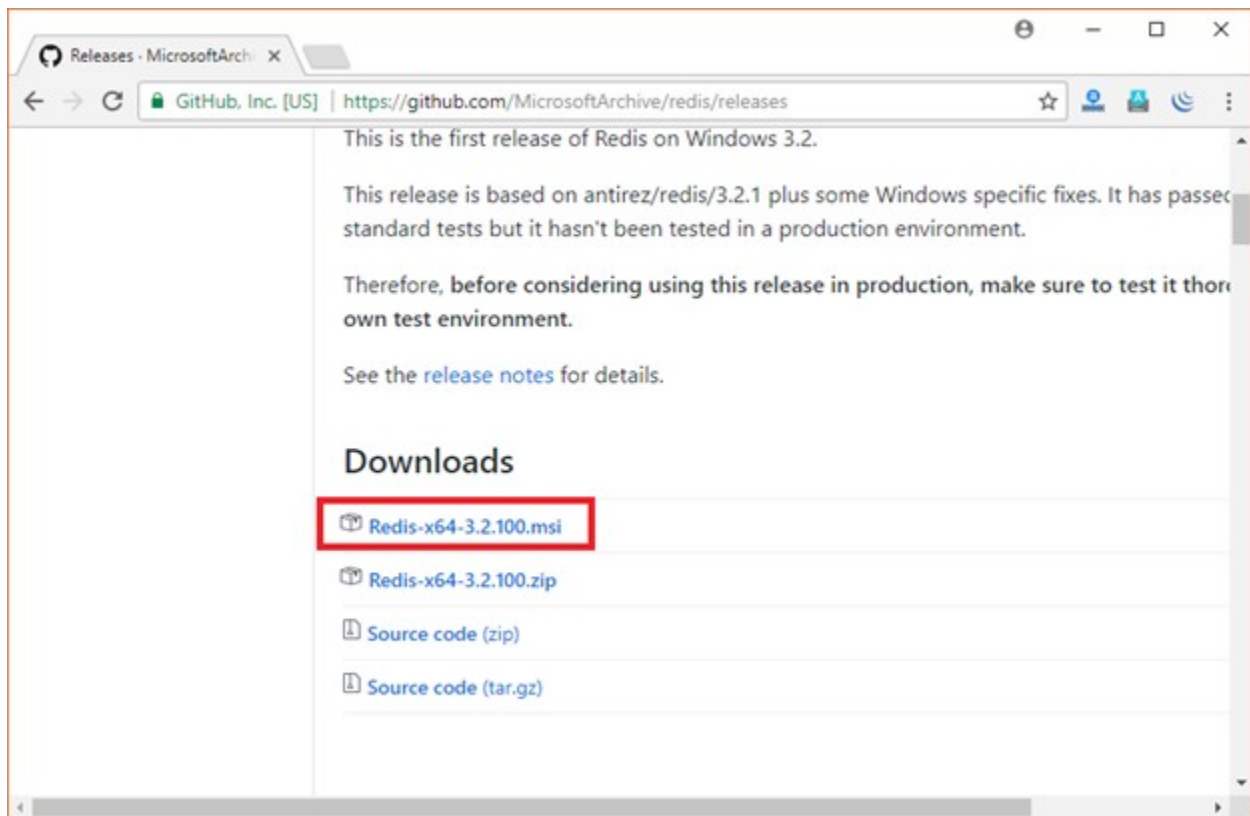
Scroll down to Windows section. You will see learn more. Click on it.



You will get release page:



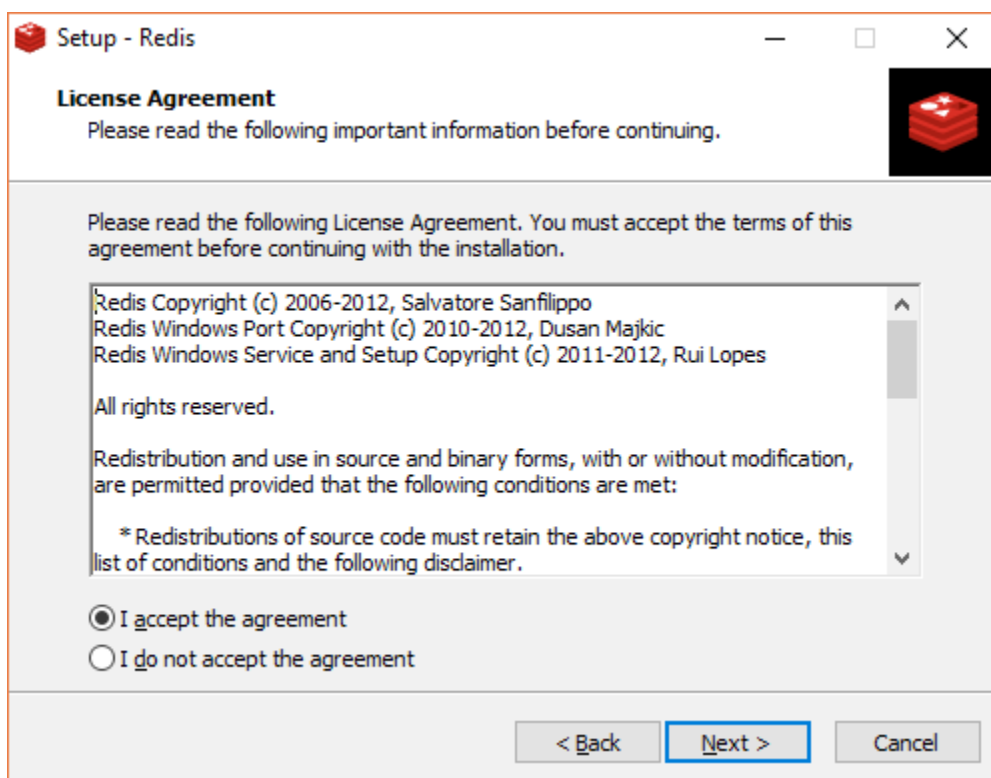
Click on download:

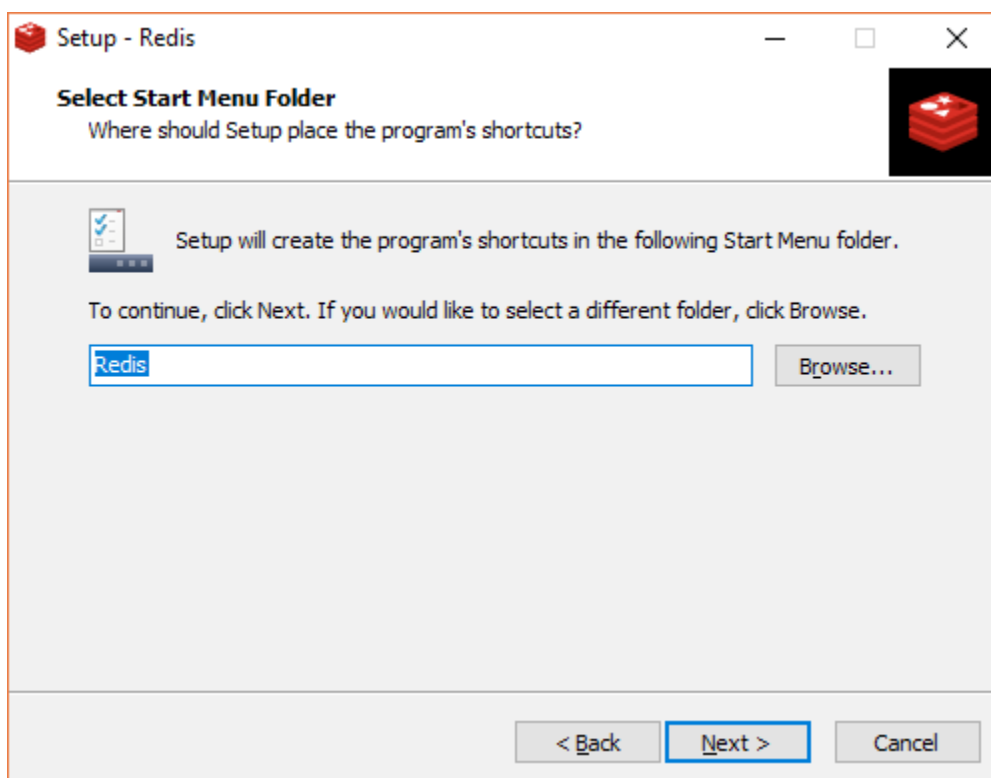
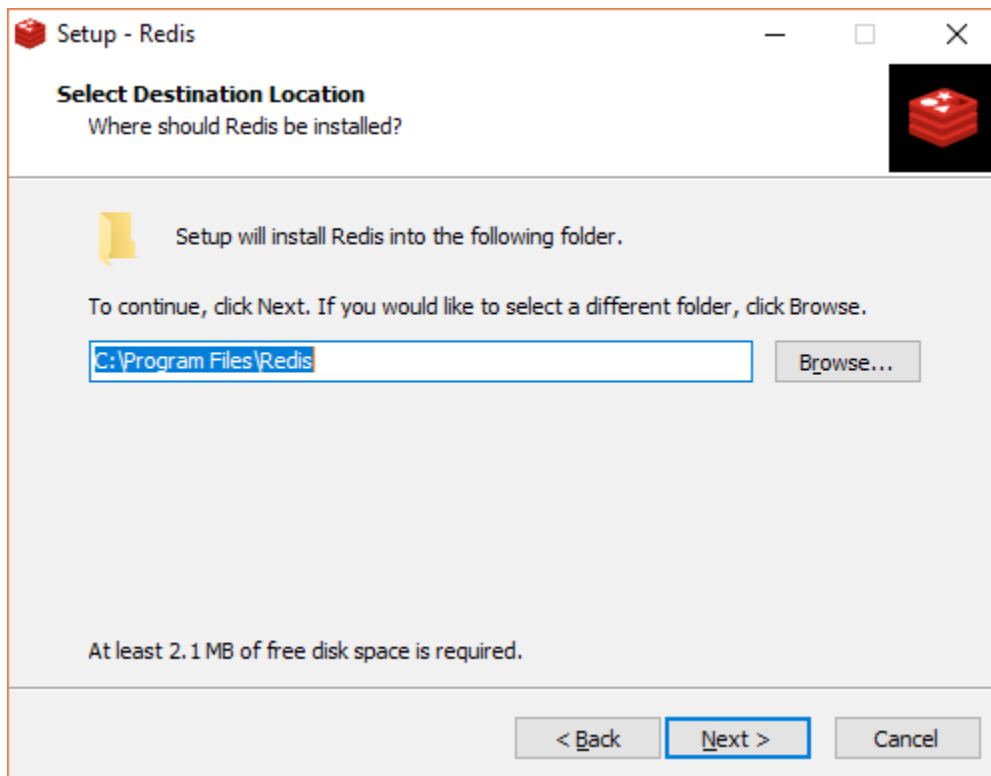


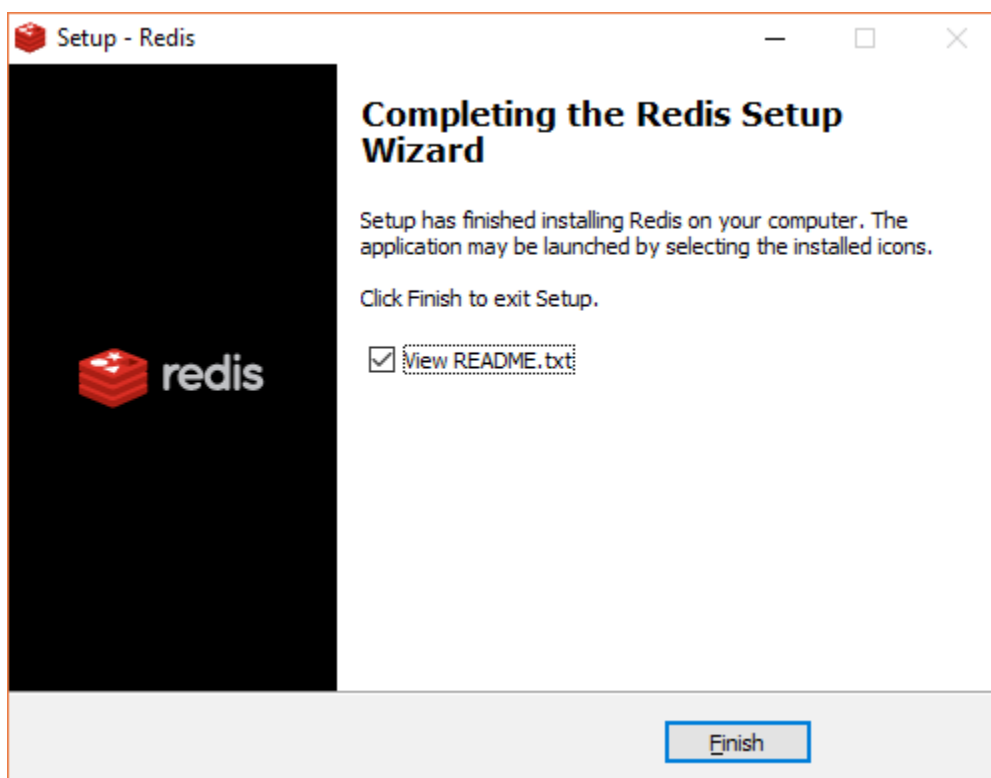
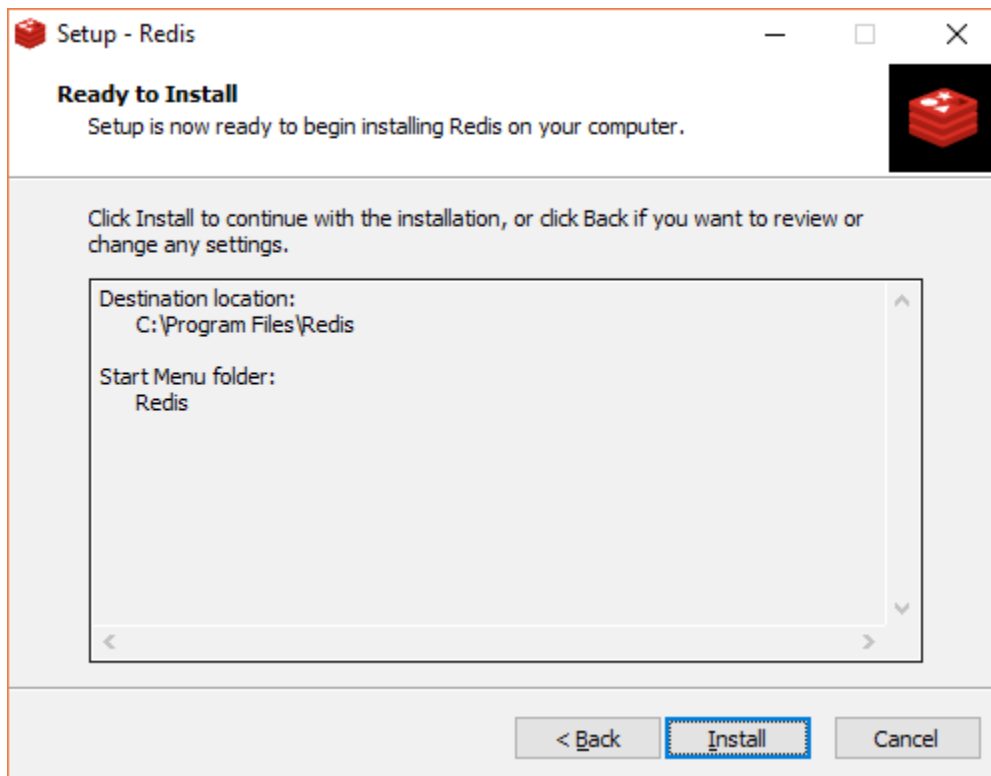
You can see that Redis is downloaded now.

You can also download one click Redis and install as a Windows service by using the following Github link.

<https://github.com/rgl/redis/downloads>

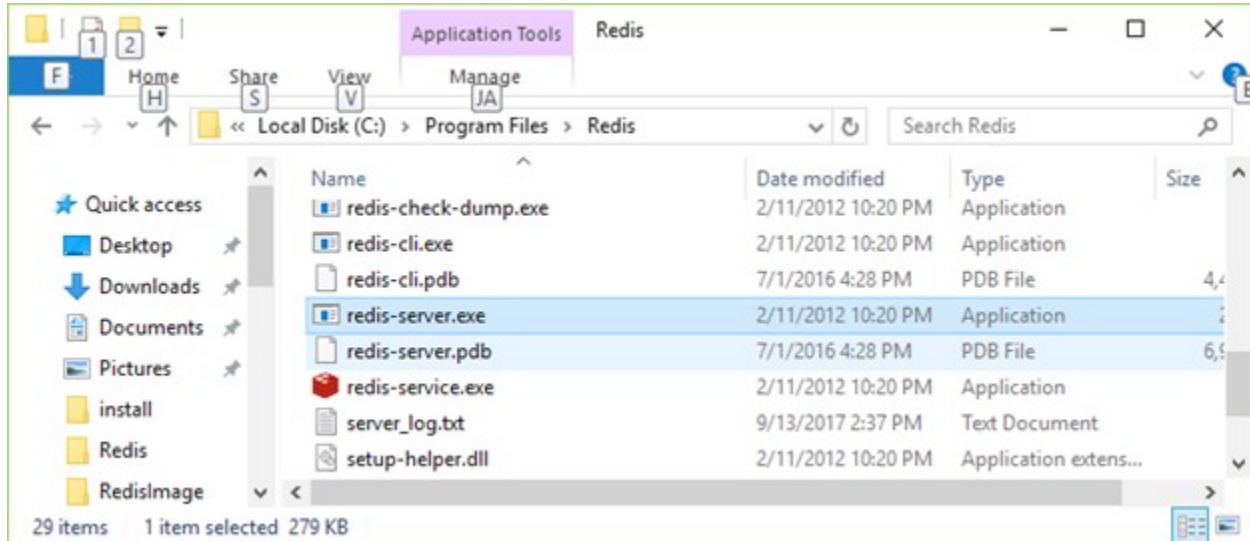




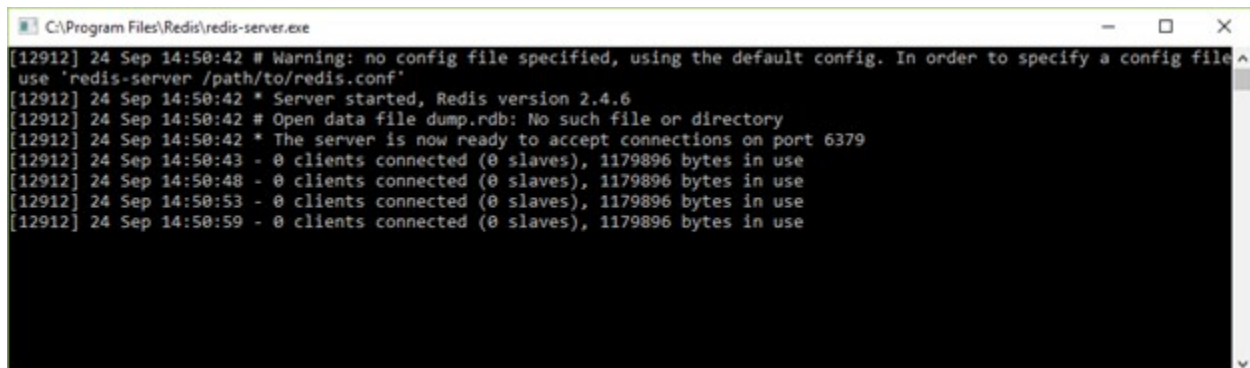


Start Redis

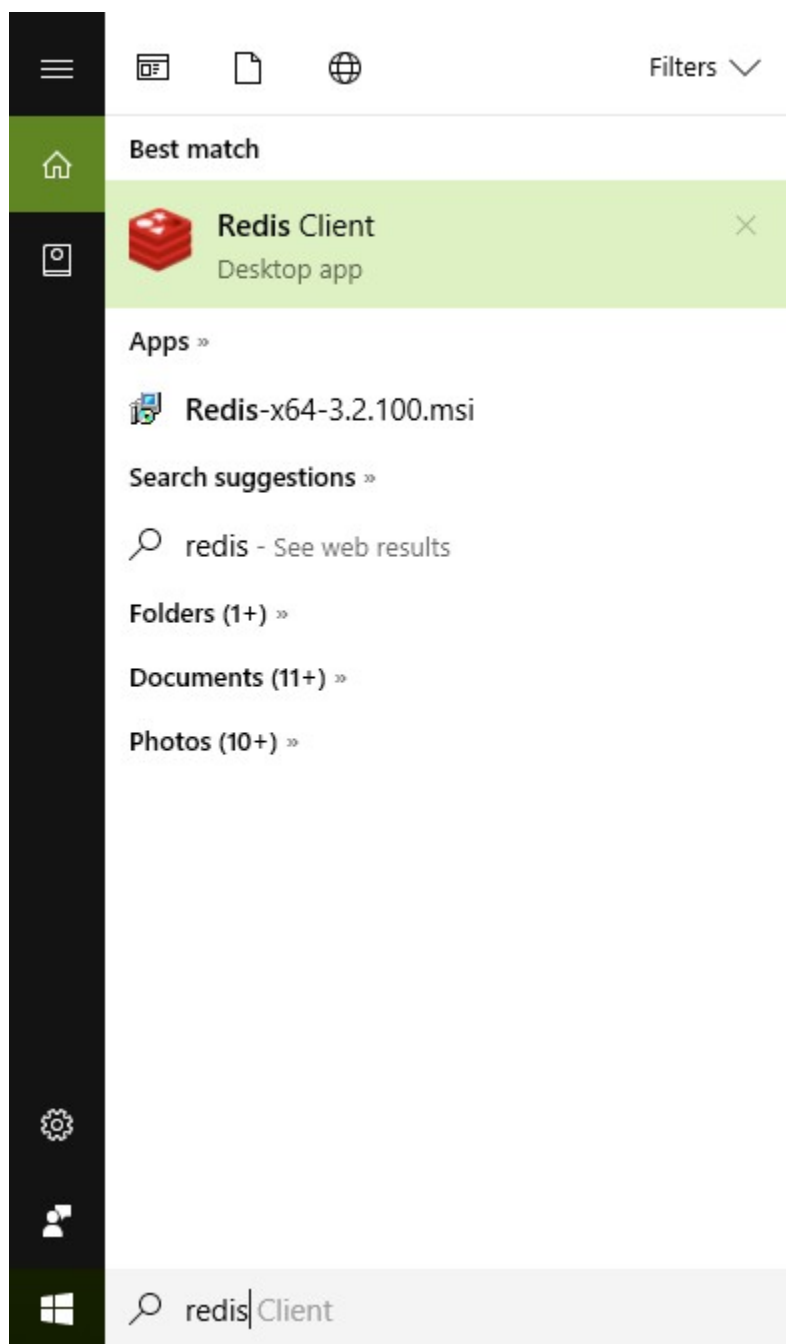
Redis is now ready to use. Start Redis server. Go to program files, followed by redis, followed by:

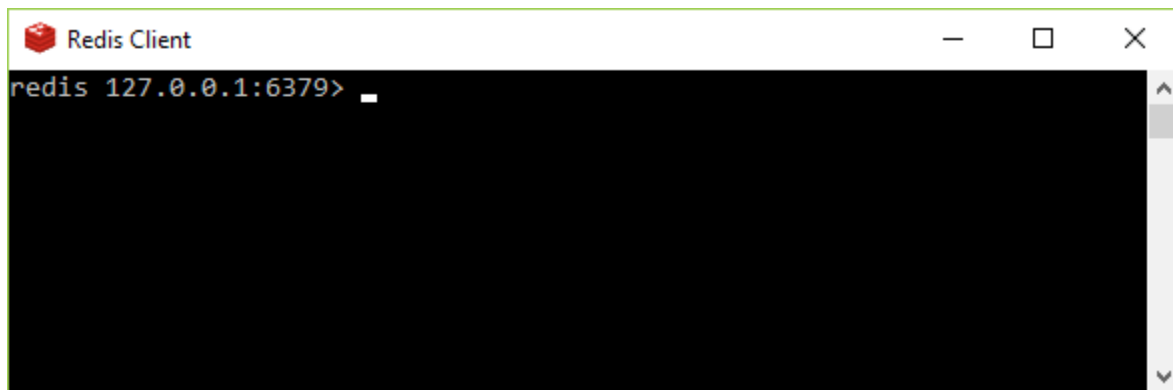


Click on the redis-server.exe and you will find that the server is started.



Now start Redis client.

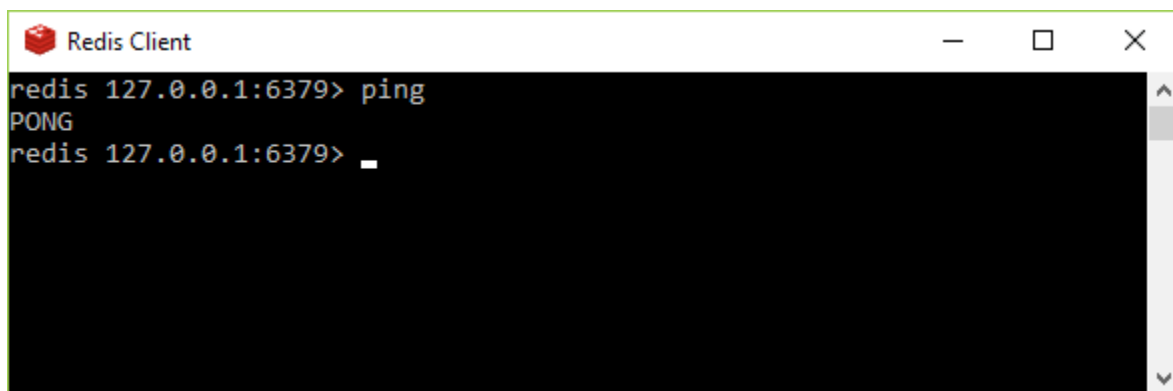




```
Redis Client
redis 127.0.0.1:6379> 
```

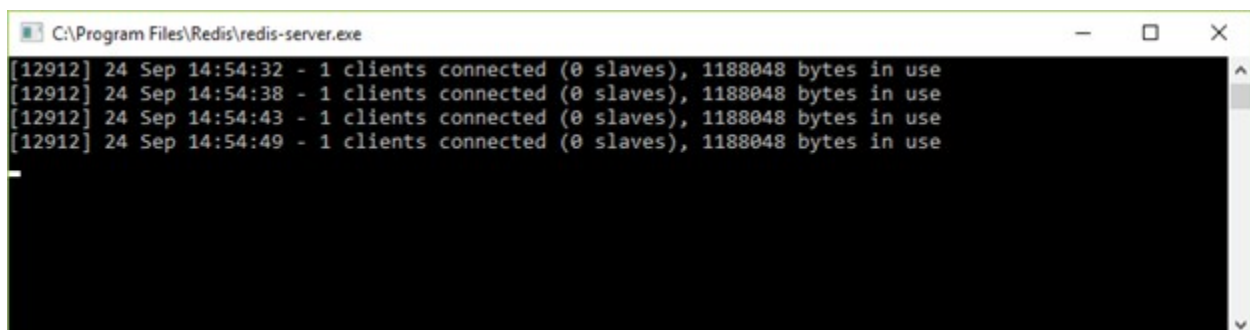
Redis is started. Now you can check whether it is connected.

Use PING command.



```
Redis Client
redis 127.0.0.1:6379> ping
PONG
redis 127.0.0.1:6379> 
```

You can see the Redis server. It will also specify that 1 client is connected.



```
C:\Program Files\Redis\redis-server.exe
[12912] 24 Sep 14:54:32 - 1 clients connected (0 slaves), 1188048 bytes in use
[12912] 24 Sep 14:54:38 - 1 clients connected (0 slaves), 1188048 bytes in use
[12912] 24 Sep 14:54:43 - 1 clients connected (0 slaves), 1188048 bytes in use
[12912] 24 Sep 14:54:49 - 1 clients connected (0 slaves), 1188048 bytes in use

```

Next Topic [Redis Installation on Ubuntu](#)

Install Redis on Ubuntu

Follow the steps given below to install Redis on Ubuntu:

First setup a non-root user using sudo and then install build and test dependencies:

1. `sudo apt update`
2. `sudo apt full-upgrade`
3. `sudo apt install build-essential tcl`

```
sssit@sssit-Inspiron-15-3567:~$ sudo apt update
[sudo] password for sssit:
Ign:1 http://dl.google.com/linux/chrome/deb stable InRelease
Hit:2 http://dl.google.com/linux/chrome/deb stable Release
Hit:4 http://security.ubuntu.com/ubuntu xenial-security InRelease
Hit:5 http://ppa.launchpad.net/jonathonf/firefox-esr/ubuntu xenial InRelease
Hit:6 http://archive.ubuntu.com/ubuntu xenial InRelease
Ign:7 http://dell.archive.canonical.com/updates xenial-dell InRelease
Hit:8 http://archive.canonical.com/ubuntu xenial InRelease
Hit:9 http://dell.archive.canonical.com/updates xenial-dell Release
Hit:10 http://archive.ubuntu.com/ubuntu xenial-updates InRelease
Hit:12 http://archive.ubuntu.com/ubuntu xenial-backports InRelease
Reading package lists... Done
Building dependency tree
Reading state information... Done
111 packages can be upgraded. Run 'apt list --upgradable' to see them.
sssit@sssit-Inspiron-15-3567:~$
```

```
sssit@sssit-Inspiron-15-3567:~$ apt list --upgradable
google-chrome-stable gvfs gvfs-backends gvfs-bin gvfs-common gvfs-daemons
gvfs-fuse gvfs-libs libapparmor-perl libapparmor1 libbind9-140 libbluetooth3
libcryptsetup4 libcups2 libcups2-cgi libcupsimage2 libcupsmime1 libcupsppdc1
libdns-export162 libdns162 libgd3 libgdk-pixbuf2.0-0 libgdk-pixbuf2.0-common
libgs9 libgs9-common libisc-export160 libisc160 libisccc140 libisccfg140
libkf5archive5 liblouis-data liblouis9 liblwres141 libpackagekit-glib2-16
libplist3 libplymouth4 libpulse-mainloop-glib0 libpulse0 libpulsedsp
libpython3.5 libpython3.5-minimal libpython3.5-stdlib libsmbclient
libsnapd-glib1 libwbclient0 libxml2 linux-firmware linux-generic
linux-headers-generic linux-image-generic linux-libc-dev
linux-signed-generic linux-signed-image-generic linux-tools-common
linux-tools-virtual plymouth plymouth-label plymouth-theme-ubuntu-logo
plymouth-theme-ubuntu-text pulseaudio pulseaudio-module-bluetooth
pulseaudio-module-x11 pulseaudio-utils python-samba python3-jwt
python3-louis python3-update-manager python3.5 python3.5-minimal
samba-common samba-common-bin samba-libs snapd snapd-login-service tcpdump
thunderbird thunderbird-gnome-support thunderbird-locales-en
thunderbird-locales-en-us ubuntu-core-launcher ubuntu-software update-manager
update-manager-core update-notifier update-notifier-common xserver-common
xserver-xorg-core
111 upgraded, 7 newly installed, 0 to remove and 0 not upgraded.
Need to get 324 MB of archives.
After this operation, 309 MB of additional disk space will be used.
Do you want to continue? [Y/n]
```

To continue press Y


```

amd64 9.18~dfsg~0-0ubuntu2.7 [34.2 kB]
Get:17 http://archive.ubuntu.com/ubuntu xenial-updates/main amd64 libgs9-commo
all 9.18~dfsg~0-0ubuntu2.7 [2,982 kB]
Get:18 http://archive.ubuntu.com/ubuntu xenial-updates/main amd64 libgs9 amd64
.18~dfsg~0-0ubuntu2.7 [2,057 kB]
Get:19 http://archive.ubuntu.com/ubuntu xenial-updates/main amd64 cups-common
l 2.1.3-4ubuntu0.3 [134 kB]
Get:20 http://archive.ubuntu.com/ubuntu xenial-updates/main amd64 cups-ppdc ar
4 2.1.3-4ubuntu0.3 [26.7 kB]
Get:21 http://archive.ubuntu.com/ubuntu xenial-updates/main amd64 cups-bsd amd
2.1.3-4ubuntu0.3 [34.8 kB]
Get:22 http://archive.ubuntu.com/ubuntu xenial-updates/main amd64 cups-client
d64 2.1.3-4ubuntu0.3 [133 kB]
Get:23 http://archive.ubuntu.com/ubuntu xenial-updates/main amd64 libcups2 amd
2.1.3-4ubuntu0.3 [197 kB]
Get:24 http://archive.ubuntu.com/ubuntu xenial-updates/main amd64 cups amd64 2
.3-4ubuntu0.3 [192 kB]
Get:25 http://archive.ubuntu.com/ubuntu xenial-updates/main amd64 libgdk-pixbu
.0-0 amd64 2.32.2-1ubuntu1.3 [159 kB]
Get:26 http://archive.ubuntu.com/ubuntu xenial-updates/main amd64 libgdk-pixbu
.0-common all 2.32.2-1ubuntu1.3 [10.4 kB]
Get:27 http://archive.ubuntu.com/ubuntu xenial-updates/universe amd64 chromiun
rowser amd64 61.0.3163.79-0ubuntu0.16.04.1300 [59.3 MB]
12% [27 chromium-browser 1,537 kB/59.3 MB 3%] [1 google-chrome-stable 15.9 MB,

```

Install Redis Server

Use the following command to install Redis server:

1. `sudo apt-get install redis-server`

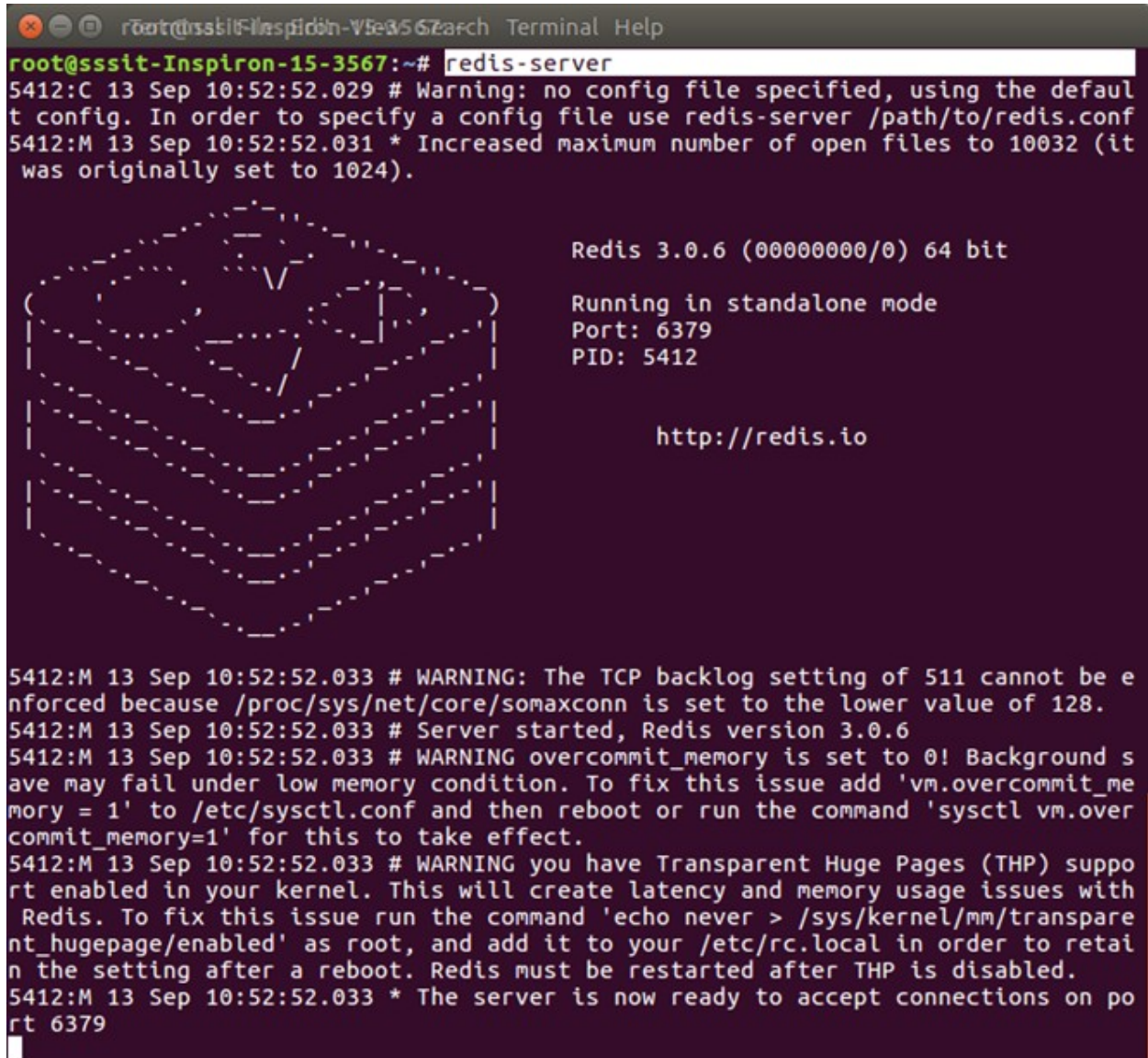
```
sssit@sssit-Inspiron-15-3567:~$ sudo apt-get install redis-server
...
nome-support amd64 1:52.3.0+build1-0ubuntu0.16.04.1 [8,544 B]
Get:113 http://archive.ubuntu.com/ubuntu xenial-updates/main amd64 thunderbird
ocale-en-us all 1:52.3.0+build1-0ubuntu0.16.04.1 [9,786 B]
Get:114 http://archive.ubuntu.com/ubuntu xenial-updates/main amd64 xserver-con
n all 2:1.18.4-0ubuntu0.4 [27.2 kB]
Get:115 http://archive.ubuntu.com/ubuntu xenial-updates/main amd64 xserver-xor
core amd64 2:1.18.4-0ubuntu0.4 [1,343 kB]
Get:116 http://archive.ubuntu.com/ubuntu xenial-updates/main amd64 liblouis-da
all 2.6.4-2ubuntu0.1 [1,077 kB]
Get:117 http://archive.ubuntu.com/ubuntu xenial-updates/main amd64 liblouis9 a
64 2.6.4-2ubuntu0.1 [59.5 kB]
Get:118 http://archive.ubuntu.com/ubuntu xenial-updates/main amd64 python3-lo
all 2.6.4-2ubuntu0.1 [7,384 B]
Fetched 324 MB in 15min 37s (346 kB/s)
Extracting templates from packages: 100%
Preconfiguring packages ...
(Reading database ... 251555 files and directories currently installed.)
Preparing to unpack .../libpython3.5_3.5.2-2ubuntu0~16.04.2_amd64.deb ...
Unpacking libpython3.5:amd64 (3.5.2-2ubuntu0~16.04.2) over (3.5.2-2ubuntu0~16
.1) ...
Preparing to unpack .../python3.5_3.5.2-2ubuntu0~16.04.2_amd64.deb ...
Unpacking python3.5 (3.5.2-2ubuntu0~16.04.2) over (3.5.2-2ubuntu0~16.04.1) ...
Progress: [ 0%] [.....]
sssit@sssit-Inspiron-15-3567:~$ sudo apt-get install redis-server
libkf5sonnetui5 libkf5textwidgets-data libkf5textwidgets5 libkf5wallet-bin
libkf5wallet-data libkf5wallet5 libkf5waylandclient5 libkf5waylandserver5
libkf5widgetsaddons-data libkf5widgetsaddons5 libkf5windowsystem-data
libkf5windowsystem5 libkf5xmlgui-bin libkf5xmlgui-data libkf5xmlgui5
libkscreenlocker5 libkwalletbackend5-5 libkwin4-effect-builtins1
libkwineffects7 libkwinglutils7 libkwinxrenderutils7 liblvm2app2.2
liblvm2cmd2.02 liborbit-2-0 libparted-fs-resize0 libphonon4qt5-4
libpolkit-qt5-1-1 libqt4-designer libqt4-help libqt4-opengl
libqt4-scripttools libqt4-svg libqt4-test libqt5script5 libqt5x11extras5
libqtassistantclient4 libqtwebkit4 libreadline5 libvoikko1 libxcb-composite
libxcb-cursor0 libxcb-damage0 linux-headers-4.4.0-23
linux-headers-4.4.0-23-generic linux-headers-4.4.0-40
linux-headers-4.4.0-40-generic linux-image-4.4.0-23-generic
linux-image-4.4.0-40-generic linux-image-extra-4.4.0-23-generic
linux-image-extra-4.4.0-40-generic linux-signed-image-4.4.0-23-generic
linux-signed-image-4.4.0-40-generic linux-tools-4.4.0-23
linux-tools-4.4.0-23-generic lvm2 plasma-framework python3-icu python3-pam
python3-pyqt4 python3-sip qml-module-org-kde-kquickcontrols
qml-module-org-kde-kquickcontrolsaddons qml-module-qtquick-controls
qml-module-qtquick-dialogs qml-module-qtquick-privatewidgets rdate
sonnet-plugins
Use 'sudo apt autoremove' to remove them.
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
sssit@sssit-Inspiron-15-3567:~$
```

Now Redis Server is installed. You can start the Redis server:

Start Redis Server

You the following command to start redis server:

1. `redis-server`



```
root@sssit-Inspiron-15-3567:~# redis-server
5412:C 13 Sep 10:52:52.029 # Warning: no config file specified, using the default
t config. In order to specify a config file use redis-server /path/to/redis.conf
5412:M 13 Sep 10:52:52.031 * Increased maximum number of open files to 10032 (it
was originally set to 1024).

Redis 3.0.6 (00000000/0) 64 bit

Running in standalone mode
Port: 6379
PID: 5412

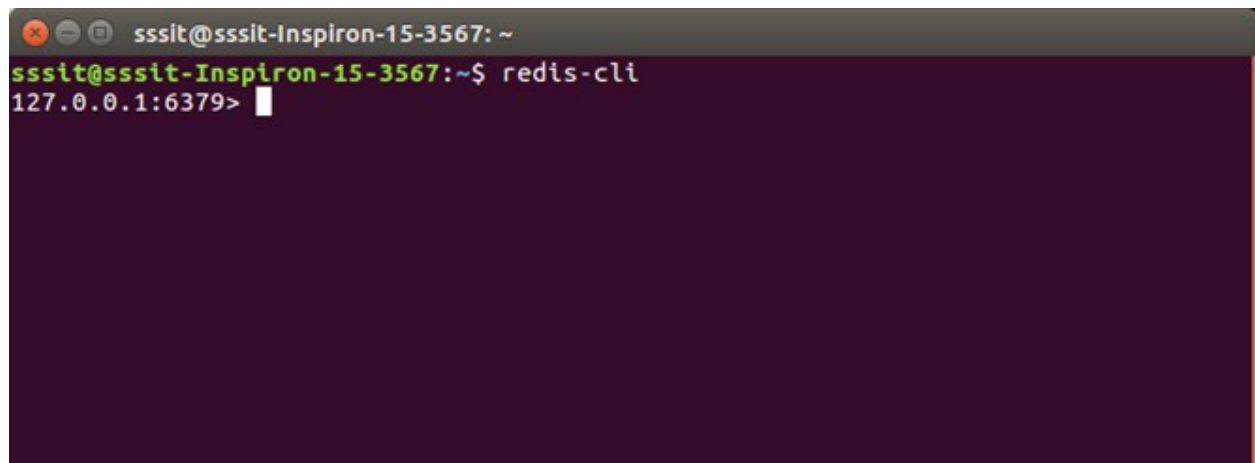
http://redis.io

5412:M 13 Sep 10:52:52.033 # WARNING: The TCP backlog setting of 511 cannot be e
nforced because /proc/sys/net/core/somaxconn is set to the lower value of 128.
5412:M 13 Sep 10:52:52.033 # Server started, Redis version 3.0.6
5412:M 13 Sep 10:52:52.033 # WARNING overcommit_memory is set to 0! Background s
ave may fail under low memory condition. To fix this issue add 'vm.overcommit_me
mory = 1' to /etc/sysctl.conf and then reboot or run the command 'sysctl vm.over
commit_memory=1' for this to take effect.
5412:M 13 Sep 10:52:52.033 # WARNING you have Transparent Huge Pages (THP) suppo
rt enabled in your kernel. This will create latency and memory usage issues with
Redis. To fix this issue run the command 'echo never > /sys/kernel/mm/transpare
nt_hugepage/enabled' as root, and add it to your /etc/rc.local in order to retai
n the setting after a reboot. Redis must be restarted after THP is disabled.
5412:M 13 Sep 10:52:52.033 * The server is now ready to accept connections on po
rt 6379
```

Start Redis Client

Redis server is started so you can start redis client to make communication between them.

1. `redis-cli`

A terminal window with a dark background. The title bar shows 'sssit@sssit-Inspiron-15-3567: ~'. The prompt is 'sssit@sssit-Inspiron-15-3567:~\$'. The command 'redis-cli' has been entered, and the prompt has changed to '127.0.0.1:6379>'.

```
sssit@sssit-Inspiron-15-3567: ~  
sssit@sssit-Inspiron-15-3567:~$ redis-cli  
127.0.0.1:6379>
```

Verify if Redis is working

Execute the following command:

1. `redis-cli`

This will open a redis prompt.

redis 127.0.0.1:6379>

In the above prompt, 127.0.0.1 is machine's IP address and 6379 is the port on which Redis server is running.

Now type the following PING command.

It specifies that Redis is successfully installed on your system.


```
sssit@sssit-Inspiron-15-3567:~$ redis-cli
127.0.0.1:6379> ping
PONG
127.0.0.1:6379> 
```

Next Topic [Redis Configuration](#)

Redis Configuration

In Redis, there is a configuration file (redis.conf) available at the root directory of Redis. Although you can get and set all Redis configurations by Redis CONFIG command.

Syntax

Following is the basic syntax of Redis CONFIG command.

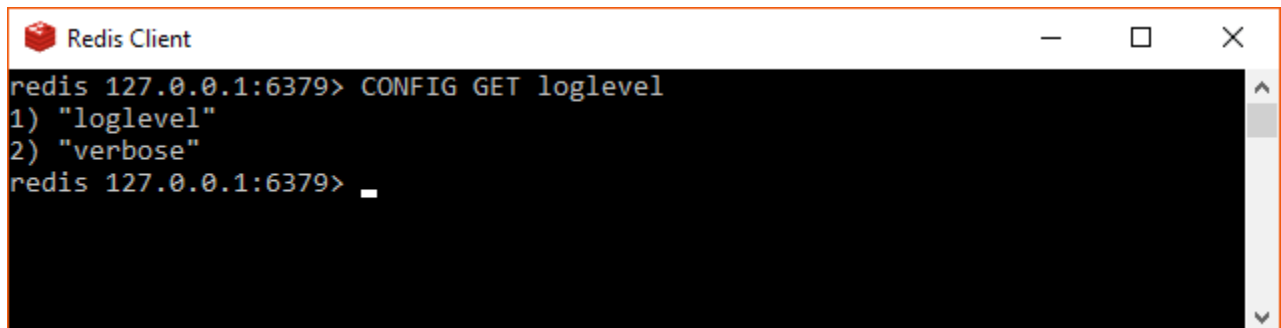
1. redis 127.0.0.1:6379> CONFIG GET CONFIG_SETTING_NAME

Example

1. redis 127.0.0.1:6379> CONFIG GET loglevel

Output

1. "loglevel"
2. "notice"

A screenshot of a Redis Client window. The title bar says "Redis Client". The terminal shows the command "redis 127.0.0.1:6379> CONFIG GET loglevel" and the output "1) \"loglevel\" 2) \"notice\"". The prompt "redis 127.0.0.1:6379> " is followed by a cursor.

```
Redis Client
redis 127.0.0.1:6379> CONFIG GET loglevel
1) "loglevel"
2) "notice"
redis 127.0.0.1:6379> _
```

To get all configuration settings, use * in place of CONFIG_SETTING_NAME

Example

1. redis 127.0.0.1:6379> CONFIG GET *

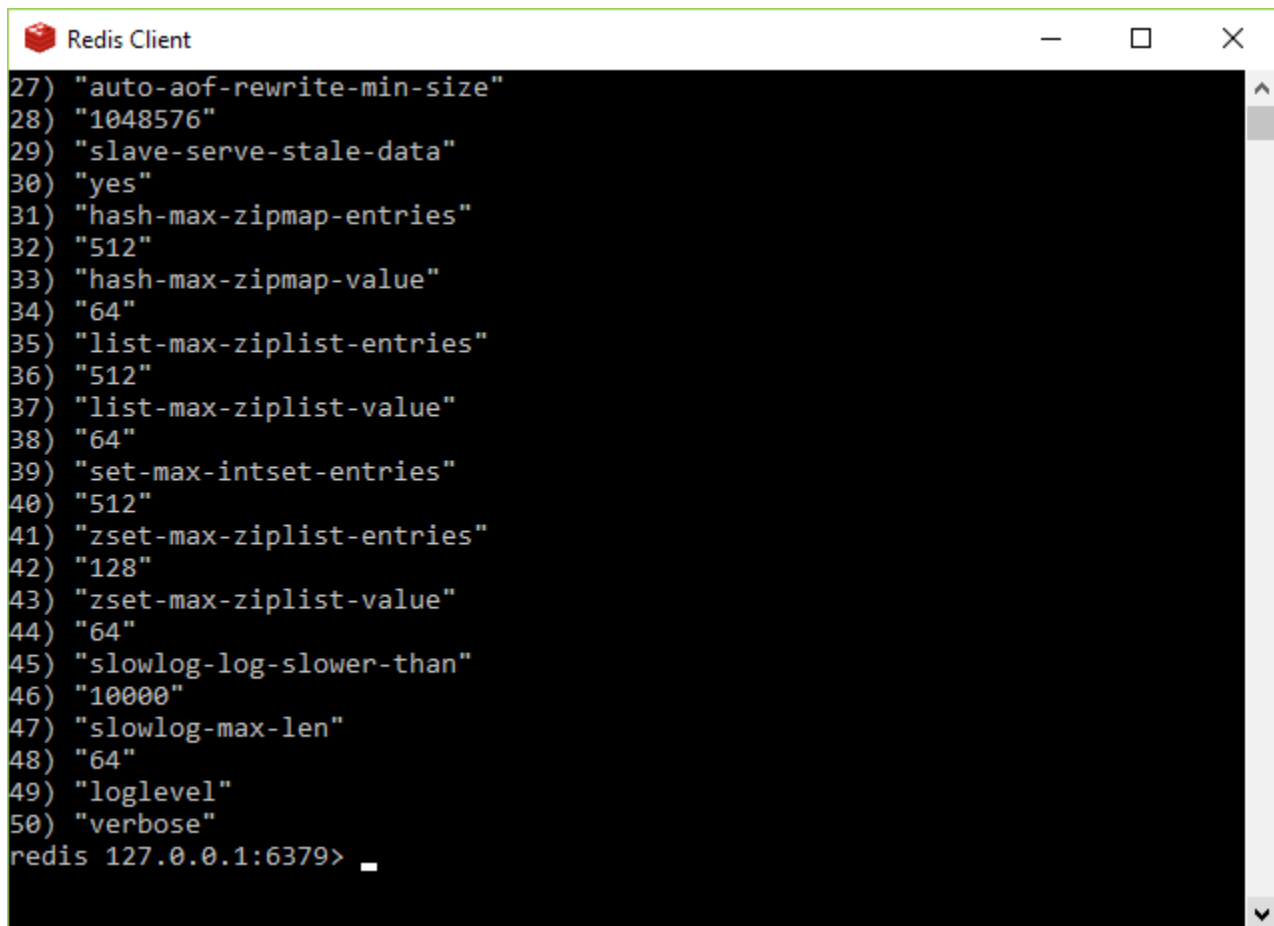
Output

1. "dir"
2. "C:\\Program Files\\Redis"
3. "dbfilename"
4. "dump.rdb"
5. "requirepass"
6. (nil)

7. "masterauth"
8. (nil)
9. "maxmemory"
10. "0"
11. "maxmemory-policy"
12. "volatile-lru"
13. "maxmemory-samples"
14. "3"
15. "timeout"
16. "0"
17. "appendonly"
18. "no"
19. "no-appendfsync-on-rewrite"
20. "no"
21. "appendfsync"
22. "everysec"
23. "save"
24. "3600 1 300 100 60 10000"
25. "auto-aof-rewrite-percentage"
26. "100"
27. "auto-aof-rewrite-min-size"
28. "1048576"
29. "slave-serve-stale-data"
30. "yes"
31. "hash-max-zipmap-entries"
32. "512"
33. "hash-max-zipmap-value"
34. "64"
35. "list-max-ziplist-entries"
36. "512"
37. "list-max-ziplist-value"
38. "64"
39. "set-max-intset-entries"
40. "512"

- 41. "zset-max-ziplist-entries"
- 42. "128"
- 43. "zset-max-ziplist-value"
- 44. "64"
- 45. "slowlog-log-slower-than"
- 46. "10000"
- 47. "slowlog-max-len"
- 48. "64"
- 49. "loglevel"
- 50. "verbose"

```
Redis Client
redis 127.0.0.1:6379> CONFIG GET *
1) "dir"
2) "C:\\Program Files\\Redis"
3) "dbfilename"
4) "dump.rdb"
5) "requirepass"
6) (nil)
7) "masterauth"
8) (nil)
9) "maxmemory"
10) "0"
11) "maxmemory-policy"
12) "volatile-lru"
13) "maxmemory-samples"
14) "3"
15) "timeout"
16) "0"
17) "appendonly"
18) "no"
19) "no-appendfsync-on-rewrite"
20) "no"
21) "appendfsync"
22) "everysec"
23) "save"
24) "3600 1 300 100 60 10000"
25) "auto-aof-rewrite-percentage"
26) "100"
```

A screenshot of a Redis Client terminal window. The window has a title bar with a red cube icon and the text "Redis Client". The terminal content shows a series of configuration commands and their values, numbered 27 to 50. The commands are: "auto-aof-rewrite-min-size", "1048576", "slave-serve-stale-data", "yes", "hash-max-ziplist-entries", "512", "hash-max-ziplist-value", "64", "list-max-ziplist-entries", "512", "list-max-ziplist-value", "64", "set-max-intset-entries", "512", "zset-max-ziplist-entries", "128", "zset-max-ziplist-value", "64", "slowlog-log-slower-than", "10000", "slowlog-max-len", "64", "loglevel", and "verbose". The prompt "redis 127.0.0.1:6379>" is at the bottom.

```
27) "auto-aof-rewrite-min-size"
28) "1048576"
29) "slave-serve-stale-data"
30) "yes"
31) "hash-max-ziplist-entries"
32) "512"
33) "hash-max-ziplist-value"
34) "64"
35) "list-max-ziplist-entries"
36) "512"
37) "list-max-ziplist-value"
38) "64"
39) "set-max-intset-entries"
40) "512"
41) "zset-max-ziplist-entries"
42) "128"
43) "zset-max-ziplist-value"
44) "64"
45) "slowlog-log-slower-than"
46) "10000"
47) "slowlog-max-len"
48) "64"
49) "loglevel"
50) "verbose"
redis 127.0.0.1:6379> _
```

Edit Configuration

To update configuration, you can edit redis.conf file directly or you can update configurations via CONFIG set command.

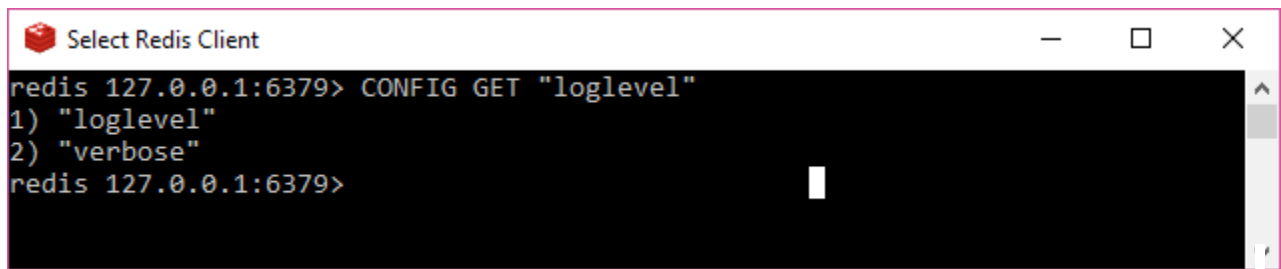
Syntax

Following is the basic syntax of CONFIG SET command.

1. `redis 127.0.0.1:6379> CONFIG SET CONFIG_SETTING_NAME NEW_CONFIG_VALUE`

Example

1. `CONFIG GET "loglevel"`



```
redis 127.0.0.1:6379> CONFIG GET "loglevel"
1) "loglevel"
2) "verbose"
redis 127.0.0.1:6379>
```

Next Topic [Redis Data Types](#)

Redis Data Type

There are five types of data types supported by Redis database.

- Strings
 - Hashes
 - Lists
 - Sets
 - Sorted Sets
-

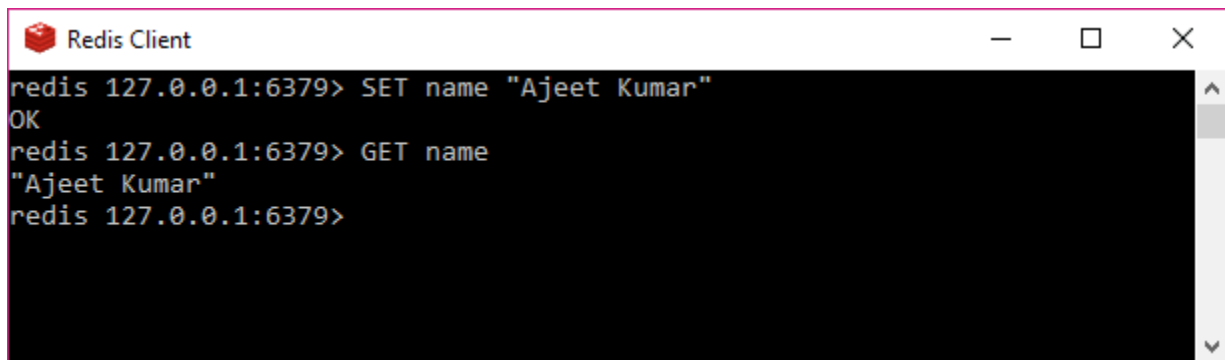
Strings

String is a set of bytes. In Redis database, strings are binary safe. It means they have a known length and not determined by any special terminating characters. So it is possible to store anything up to 512 megabytes in one string.

Example

Let's store a string name "Ajeet Kumar" in the key by using SET command and then retrieve the same by using GET command.

1. SET name "Ajeet Kumar"
2. OK
3. GET name
4. "Ajeet Kumar"

A screenshot of a terminal window titled "Redis Client". The terminal shows a series of commands and responses. The first command is "redis 127.0.0.1:6379> SET name 'Ajeet Kumar'", followed by the response "OK". The second command is "redis 127.0.0.1:6379> GET name", followed by the response "'Ajeet Kumar'". The prompt "redis 127.0.0.1:6379>" is shown again at the bottom, indicating the session is still active. The terminal has a black background with white text, and a scrollbar is visible on the right side.

```
redis 127.0.0.1:6379> SET name "Ajeet Kumar"
OK
redis 127.0.0.1:6379> GET name
"Ajeet Kumar"
redis 127.0.0.1:6379>
```

In the above example, SET and GET are the Redis command, name is the key used in Redis, "Ajeet Kumar" is string value stored in Redis.

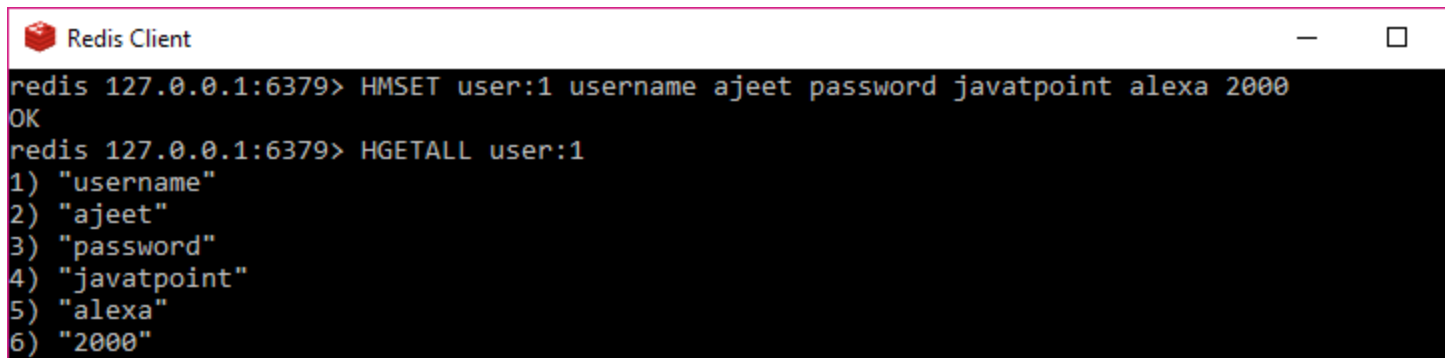
Hashes

Hash is a collection of key-value pairs. In Redis, hashes are maps between string fields and string values. So, they are used to represent objects.

Example

Let's store a user's object which contains basic information of a user.

1. HMSET user:1 username ajeet password javatpoint alexa 2000
2. OK
3. HGETALL user:1
4. "username"
5. "ajeet"
6. "password"
7. "javatpoint"
8. "alexa"
9. "2000"



```
Redis Client
redis 127.0.0.1:6379> HMSET user:1 username ajeet password javatpoint alexa 2000
OK
redis 127.0.0.1:6379> HGETALL user:1
1) "username"
2) "ajeet"
3) "password"
4) "javatpoint"
5) "alexa"
6) "2000"
```

Here, HMSET and HGETALL are the command for Redis, while user:1 is the key.

Every hash can store up to $2^{32} - 1$ field-value pairs (more than 4 billion).

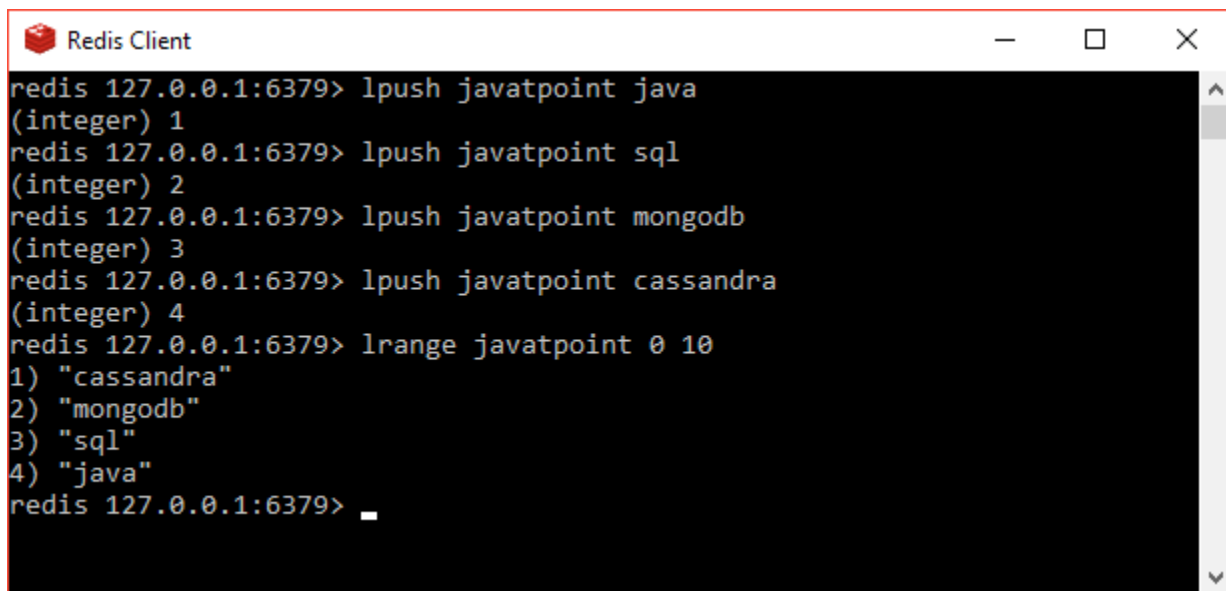
Lists

Redis Lists are defined as a lists of strings, sorted by insertion order. You can add elements to a Redis List on the head or on the tail.

Example

1. lpush javatpoint java
2. (integer) 1

3. lpush javatpoint java
4. (integer) 1
5. lpush javatpoint java
6. (integer) 1
7. lpush javatpoint java
8. (integer) 1
9. lrange javatpoint 0 10
10. "cassandra"
11. "mongodb"
12. "sql"
13. "java"

A screenshot of a Redis Client window. The window title is "Redis Client". The terminal shows the following commands and output:

```
redis 127.0.0.1:6379> lpush javatpoint java
(integer) 1
redis 127.0.0.1:6379> lpush javatpoint sql
(integer) 2
redis 127.0.0.1:6379> lpush javatpoint mongodb
(integer) 3
redis 127.0.0.1:6379> lpush javatpoint cassandra
(integer) 4
redis 127.0.0.1:6379> lrange javatpoint 0 10
1) "cassandra"
2) "mongodb"
3) "sql"
4) "java"
redis 127.0.0.1:6379> _
```

The max length of a list is $2^{32} - 1$ elements (more than 4 billion of elements per list).

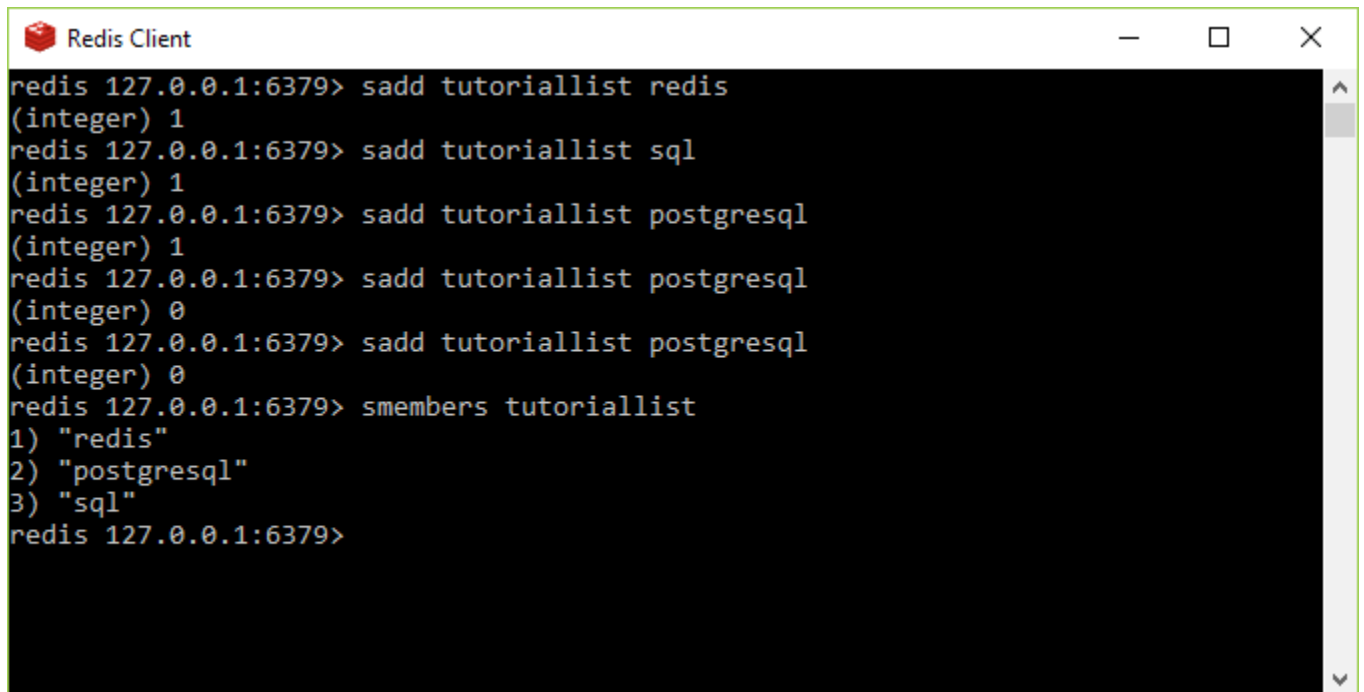
Sets

Sets are an unordered collection of strings in Redis database. In Redis, you can add, remove, and test for the existence of members in $O(1)$ time complexity.

Example

1. sadd tutoriallist redis
2. (integer) 1
3. redis 127.0.0.1:6379> sadd tutoriallist sql
4. (integer) 1

5. redis 127.0.0.1:6379> sadd tutoriallist postgresql
6. (integer) 1
7. redis 127.0.0.1:6379> sadd tutoriallist postgresql
8. (integer) 0
9. redis 127.0.0.1:6379> sadd tutoriallist postgresql
10. (integer) 0
11. redis 127.0.0.1:6379> smembers tutoriallist
12. 1) "redis"
13. 2) "postgresql"
14. 3) "sql"



```
Redis Client
redis 127.0.0.1:6379> sadd tutoriallist redis
(integer) 1
redis 127.0.0.1:6379> sadd tutoriallist sql
(integer) 1
redis 127.0.0.1:6379> sadd tutoriallist postgresql
(integer) 1
redis 127.0.0.1:6379> sadd tutoriallist postgresql
(integer) 0
redis 127.0.0.1:6379> sadd tutoriallist postgresql
(integer) 0
redis 127.0.0.1:6379> smembers tutoriallist
1) "redis"
2) "postgresql"
3) "sql"
redis 127.0.0.1:6379>
```

In the above example, you can see that postgresql is added thrice but due to unique property of the set it is added only once.

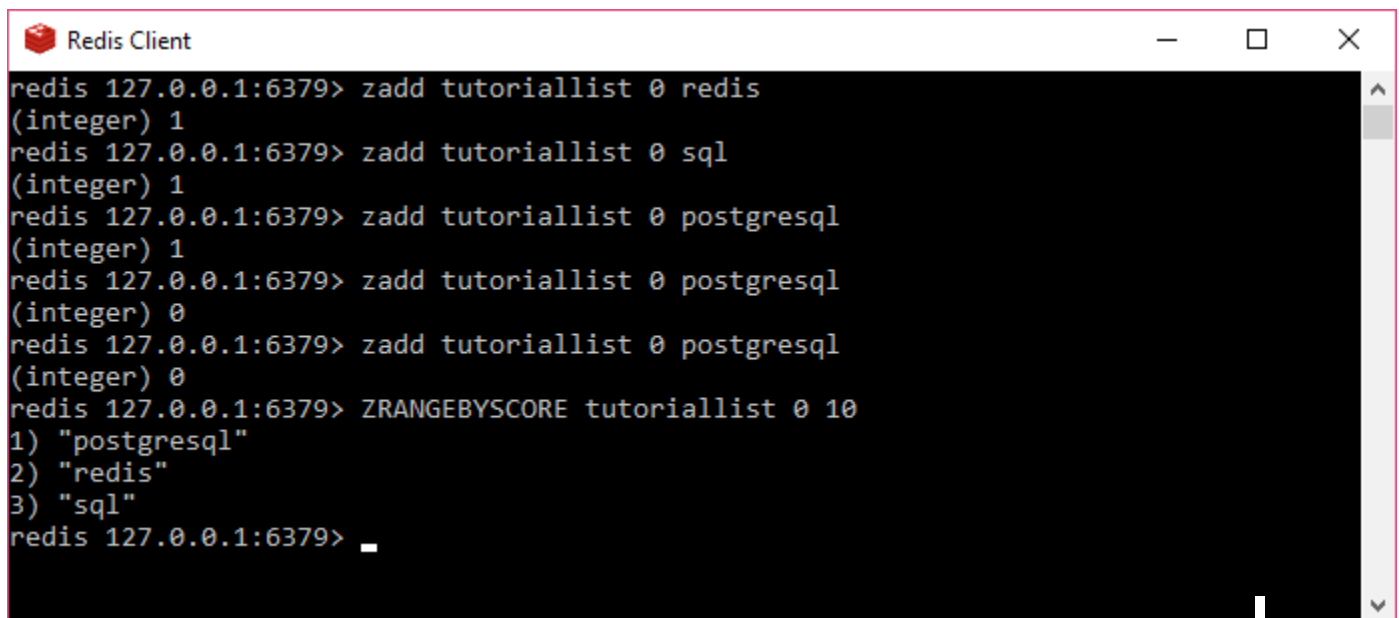
The max number of members in a set is $2^{32} - 1$ elements (more than 4 billion of elements per list).

Sorted Sets

Redis Sorted Sets are similar to Redis Sets. They are also a set of non-repeating collections of Strings. But every member of a Sorted Set is associated with a score, that is used in order to take the sorted set ordered, from the smallest to the greatest score. While members are unique, the scores may be repeated.

Example

1. redis 127.0.0.1:6379> zadd tutoriallist 0 redis
2. (integer) 1
3. redis 127.0.0.1:6379> zadd tutoriallist 0 sql
4. (integer) 1
5. redis 127.0.0.1:6379> zadd tutoriallist 0 postgresql
6. (integer) 1
7. redis 127.0.0.1:6379> zadd tutoriallist 0 postgresql
8. (integer) 0
9. redis 127.0.0.1:6379> zadd tutoriallist 0 postgresql
10. (integer) 0
11. redis 127.0.0.1:6379> ZRANGEBYSCORE tutoriallist 0 10
12. 1) "postgresql"
13. 2) "redis"
14. 3) "sql"



```
Redis Client
redis 127.0.0.1:6379> zadd tutoriallist 0 redis
(integer) 1
redis 127.0.0.1:6379> zadd tutoriallist 0 sql
(integer) 1
redis 127.0.0.1:6379> zadd tutoriallist 0 postgresql
(integer) 1
redis 127.0.0.1:6379> zadd tutoriallist 0 postgresql
(integer) 0
redis 127.0.0.1:6379> zadd tutoriallist 0 postgresql
(integer) 0
redis 127.0.0.1:6379> ZRANGEBYSCORE tutoriallist 0 10
1) "postgresql"
2) "redis"
3) "sql"
redis 127.0.0.1:6379> _
```

Next Topic [Redis All Commands](#)

Redis All Commands

Command	Description
APPEND key value	Append a value to a key
AUTH password	Authenticate to the server
BGREWRITEAOF	Asynchronously rewrite the append only file
BGSAVE	Asynchronously save the dataset to disk
BITCOUNT key [start end]	Count set bits in a string
BITFIELD key [GET type offset] [SET type offset value] [INCRBY type offset increment] [OVERFLOW WRAP SAT FAIL]	Perform arbitrary bitfield operations on strings
BITOP operation destkey key [key ...]	Perform bitwise operation on strings
BITPOS key bit [start] [end]	Find first bit set or clear
BLPOP key [key ...] timeout	Remove and get the first element of a list, or block until one is available
BRPOP key [key ...] timeout	Remove and get the last element of a list, or block until one is available
BRPOPLPUSH source destination timeout	Pop a value from a list, push it to another list, and return it; or block until a value is available
CLIENT KILL [ip:port] [ID client-id] [TYPE normal master slave pubsub] [ADDR ip:port] [SKIPME yes/no]	Kill the connection of a client
CLIENT LIST	Get the list of client connections
CLIENT GETNAME	Get the current connection name
CLIENT PAUSE timeout	Stop processing commands for some time

CLIENT REPLY ON OFF SKIP	Instruct the server whether to reply to commands
CLIENT SETNAME connection-name	Set the current connection name
CLUSTER ADDSLOTS slot [slot ...]	Assign new hash slots to this node
CLUSTER COUNT-FAILURE-REPORTS node-id	Return the number of failure reports for a given node
CLUSTER COUNTKEYSINSLOT slot	Return the number of keys in the specified hash slot
CLUSTER DELSLOTS slot [slot ...]	Set hash slots as unbound
CLUSTER FAILOVER [FORCE TAKEOVER]	Forces a slave to perform a failover of its master.
CLUSTER FORGET node-id	Remove a node from the cluster
CLUSTER GETKEYSINSLOT slot count	Return local key names in the specified slot
CLUSTER INFO	Provides info about Redis Cluster
CLUSTER KEYSLOT key	Returns the hash slot of a key
CLUSTER MEET ip port	Force a node cluster to meet with another node
CLUSTER NODES	Get Cluster config for the node
CLUSTER REPLICATE node-id	Reconfigure a node as a replica of a specified master node
CLUSTER RESET [HARD SOFT]	Reset a Redis Cluster node
CLUSTER SAVECONFIG	Forces the node to save the cluster configuration
CLUSTER SET-CONFIG-EPOCH config-epoch	Set the configuration epoch

CLUSTER SETSLOT slot IMPORTING MIGRATING STABLE NODE [node-id]	Bind a hash slot to a specific node
CLUSTER SLAVES node-id	List slave nodes of the specified master
CLUSTER SLOTS	Get array of Cluster slot information
COMMAND	Get array of Redis command names
COMMAND COUNT	Get total number of Redis commands
COMMAND GETKEYS	Extract keys given a full command
COMMAND INFO command-name [command-name ...]	Get array of specific Redis command information
CONFIG GET parameter	Get the value of a configuration parameter
CONFIG REWRITE	Rewrite the configuration file with the current memory configuration
CONFIG SET parameter value	Set a configuration parameter to a new value
CONFIG RESETSTAT	Reset the stats returned by CONFIG GET
DBSIZE	Return the number of keys in the currently selected database
DEBUG OBJECT key	Get debugging information about a key
DEBUG SEGFAULT	Make the server crash
DECR key	Decrement the integer value of a key by one
DECRBY key decrement	Decrement the integer value of a key by the given number
DEL key [key ...]	Delete a key
DISCARD	Discard all commands issued in a MULTI transaction

DUMP key	Return a serialized version of the value stored at the specified key
ECHO message	Echo the given string
EVAL script numkeys key [key ...] arg [arg ...]	Execute a Lua script server-side
EVALSHA sha1 numkeys key [key ...] arg [arg ...]	Execute a Lua script server-side
EXEC	Execute all commands issued in the last 24 hours
EXISTS key [key ...]	Determine if a key exists
EXPIRE key seconds	Set a key's time to live in seconds
EXPIREAT key timestamp	Set the expiration for a key as a Unix timestamp
FLUSHALL [ASYNC]	Remove all keys from all databases
FLUSHDB [ASYNC]	Remove all keys from the current database
GEOADD key longitude latitude member [longitude latitude member ...]	Add one or more geospatial items to a geospatial index represented as a sorted set
GEOHASH key member [member ...]	Returns members of a geospatial index as standard geohash strings
GEOPOS key member [member ...]	Returns longitude and latitude of members of a geospatial index
GEODIST key member1 member2 [unit]	Returns the distance between two members of a geospatial index
GEORADIUS key longitude latitude radius m km ft mi [WITHCOORD] [WITHDIST] [WITHHASH] [COUNT count] [ASC DESC] [STORE key] [STOREDIST key]	Query a sorted set representing a geospatial index to fetch members within a maximum distance from a point
GEORADIUSBYMEMBER key member radius m km ft mi [WITHCOORD]	Query a sorted set representing a geospatial index to fetch members within a maximum distance from a point

[WITHDIST] [WITHHASH] [COUNT count] [ASC DESC] [STORE key] [STOREDIST key]	index to fetch members maximum distance from
GET key	Get the value of a key
GETBIT key offset	Returns the bit value at value stored at key
GETRANGE key start end	Get a substring of the str
GETSET key value	Set the string value of a old value
HDEL key field [field ...]	Delete one or more hash
HEXISTS key field	Determine if a hash field
HGET key field	Get the value of a hash f
HGETALL key	Get all the fields and val
HINCRBY key field increment	Increment the integer va by the given number
HINCRBYFLOAT key field increment	Increment the float value the given amount
HKEYS key	Get all the fields in a has
HLEN key	Get the number of fields
HMGET key field [field ...]	Get the values of all the
HMSET key field value [field value ...]	Set multiple hash fields t
HSET key field value	Set the string value of a
HSETNX key field value	Set the value of a hash f does not exist
HSTRLEN key field	Get the length of the val

HVALS key	Get all the values in a hash
INCR key	Increment the integer value of a key
INCRBY key increment	Increment the integer value of a key by a given amount
INCRBYFLOAT key increment	Increment the float value of a key by a given amount
INFO [section]	Get information and statistics about the server
KEYS pattern	Find all keys matching the given pattern
LASTSAVE	Get the UNIX time stamp of the last successful save to disk
LINDEX key index	Get an element from a list by index
LINSERT key BEFORE AFTER pivot value	Insert an element before or after a pivot element in a list
LLEN key	Get the length of a list
LPOP key	Remove and get the first element of a list
LPUSH key value [value ...]	Prepend one or multiple values to a list
LPUSHX key value	Prepend a value to a list, if the list exists
LRANGE key start stop	Get a range of elements from a list
LREM key count value	Remove elements from a list by value
LSET key index value	Set the value of an element in a list by index
LTRIM key start stop	Trim a list to the specified range

MGET key [key ...]	Get the values of all the
MIGRATE host port key "" destination-db timeout [COPY] [REPLACE] [KEYS key [key ...]]	Atomically transfer a key instance to another one.
MONITOR	Listen for all requests received in real time
MOVE key db	Move a key to another database
MSET key value [key value ...]	Set multiple keys to multiple values
MSETNX key value [key value ...]	Set multiple keys to multiple values, if none of the keys exist
MULTI	Mark the start of a transaction
OBJECT subcommand [arguments [arguments ...]]	Inspect the internals of Redis objects
PERSIST key	Remove the expiration from a key
PEXPIRE key milliseconds	Set a key's time to live in milliseconds
PEXPIREAT key milliseconds-timestamp	Set the expiration for a key at the specified timestamp specified in milliseconds
PFADD key element [element ...]	Adds the specified element to the HyperLogLog.
PFCOUNT key [key ...]	Return the approximated cardinality of the set(s) observed by the HyperLogLog key(s).
PFMERGE destkey sourcekey [sourcekey ...]	Merge N different HyperLogLogs into one.
PING [message]	Ping the server
PSETEX key milliseconds value	Set the value and expiration of a key

PSUBSCRIBE pattern [pattern ...]	Listen for messages published to channels matching the given pattern
PUBSUB subcommand [argument [argument ...]]	Inspect the state of the Pub/Sub system
PTTL key	Get the time to live for a key
PUBLISH channel message	Post a message to a channel
PUNSUBSCRIBE [pattern [pattern ...]]	Stop listening for messages on the specified channels matching the given patterns
QUIT	Close the connection
RANDOMKEY	Return a random key from the database
READONLY	Enables read queries for a cluster slave node
READWRITE	Disables read queries for a cluster slave node
RENAME key newkey	Rename a key
RENAMENX key newkey	Rename a key, only if the new key does not exist
RESTORE key ttl serialized-value [REPLACE]	Rename a key, only if the new key does not exist
RESTORE key ttl serialized-value [REPLACE]	Create a key using the provided serialized value, previously obtained with DUMP
ROLE	Return the role of the instance in the context of replication
RPOP key	Remove and get the last element in a list
RPOPLPUSH source destination	Remove the last element from the source list and push it to another list and return it

RPUSH key value [value ...]	Append one or multiple v
RPUSHX key value	Append a value to a list,
SADD key member [member ...]	Add one or more membe
SAVE	Synchronously save the
SCARD key	Get the number of memb
SCRIPT DEBUG YES SYNC NO	Set the debug mode for
SCRIPT EXISTS sha1 [sha1 ...]	Check existence of script cache.
SCRIPT FLUSH	Remove all the scripts fr
SCRIPT KILL	Kill the script currently in
SCRIPT LOAD script	Load the specified Lua sc cache.
SDIFF key [key ...]	Subtract multiple sets
SDIFFSTORE destination key [key ...]	Subtract multiple sets an set in a key
SELECT index	Change the selected data connection
SET key value [EX seconds] [PX milliseconds] [NX XX]	Set the string value of a
SETBIT key offset value	Sets or clears the bit at o value stored at key
SETEX key seconds value	Set the value and expira
SETNX key value	Set the value of a key, o not exist
SETRANGE key offset value	Overwrite part of a string

	the specified offset
SHUTDOWN [NOSAVE SAVE]	Synchronously save the dataset to disk then shut down the server
SINTER key [key ...]	Intersect multiple sets
SINTERSTORE destination key [key ...]	Intersect multiple sets and store the resulting set in a key
SISMEMBER key member	Determine if a given value is a member of a set
SLAVEOF host port	Make the server a slave of another Redis server or promote it as master
SLOWLOG subcommand [argument]	Manages the Redis slow command log
SMEMBERS key	Get all the members in a set
SMOVE source destination member	Move a member from one set to another
SORT key [BY pattern] [LIMIT offset count] [GET pattern [GET pattern ...]] [ASC DESC] [ALPHA] [STORE destination]	Sort the elements in a list or set
SPOP key [count]	Remove and return one or more random members from a set
SRANDMEMBER key [count]	Get one or multiple random members from a set
SREM key member [member ...]	Remove one or more members from a set
STRLEN key	Get the length of the value stored in a key
SUBSCRIBE channel [channel ...]	Listen for messages published to the specified channels
SUNION key [key ...]	Add multiple sets

SUNIONSTORE destination key [key ...]	Add multiple sets and store in a key
SWAPDB index index	Swaps two Redis databases
SYNC	Internal command used
TIME	Return the current server
TOUCH key [key ...]	Alters the last access time Returns the number of elements specified.
TTL key	Get the time to live for a
TYPE key	Determine the type stored
UNSUBSCRIBE [channel [channel ...]]	Stop listening for messages on given channels
UNLINK key [key ...]	Delete a key asynchronously in a new thread. Otherwise it is just blocking.
UNWATCH	Forget about all watched keys
WAIT numslaves timeout	Wait for the synchronous replication of the write commands sent to the current connection
WATCH key [key ...]	Watch the given keys to fail the MULTI/EXEC block
ZADD key [NX XX] [CH] [INCR] score member [score member ...]	Add one or more members to a sorted set, or update its score if it already exists
ZCARD key	Get the number of members in a sorted set

ZCOUNT key min max

Count the members in a sorted set with scores within the given values

ZINCRBY key increment member

Increment the score of a member in a

ZINTERSTORE destination numkeys key
[key ...] [WEIGHTS weight [weight ...]]
[AGGREGATE SUM|MIN|MAX]

ZLEXCOUNT key min max

ZRANGE key start stop [WITHSCORES]

ZRANGEBYLEX key min max [LIMIT offset
count]

ZREVRANGEBYLEX key max min [LIMIT offset
count]

ZRANGEBYSCORE key min max
[WITHSCORES] [LIMIT offset count]

ZRANK key member

ZREM key member [member ...]

ZREMRANGEBYLEX key min max

ZREMRANGEBYRANK key start stop

ZREMRANGEBYSCORE key min max

ZREVRANGE key start stop [WITHSCORES]

ZREVRANGEBYSCORE key max min
[WITHSCORES] [LIMIT offset count]

ZREVRANK key member

sorted set

Intersect multiple sorted sets and store
the resulting sorted set in a new key

Count the number of members in a
sorted set between a given
lexicographical range

Return a range of members in a sorted
set, by index

Return a range of members in a sorted
set, by lexicographical range

Return a range of members in a sorted
set, by lexicographical range, ordered
from higher to lower strings.

Return a range of members in a sorted
set, by score

Determine the index of a member in a
sorted set

Remove one or more members from a
sorted set

Remove all members in a sorted set
between the given lexicographical
range

Remove all members in a sorted set
within the given indexes

Remove all members in a sorted set
within the given scores

Return a range of members in a sorted
set, by index, with scores ordered from
high to low

Return a range of members in a sorted
set, by score, with scores ordered from
high to low

Determine the index of a member in a
sorted set, with scores ordered from
high to low

ZSCORE key member

Get the score associated with the given member in a sorted set

ZUNIONSTORE destination numkeys key

[key ...] [WEIGHTS weight [weight ...]]

Add multiple sorted sets and store the resulting sorted set in a new key

[AGGREGATE SUM|MIN|MAX]

SCAN cursor [MATCH pattern] [COUNT count]

Incrementally iterate the keys space

SSCAN key cursor [MATCH pattern] [COUNT count]

Incrementally iterate Set elements

HSCAN key cursor [MATCH pattern] [COUNT count]

Incrementally iterate hash fields and associated values

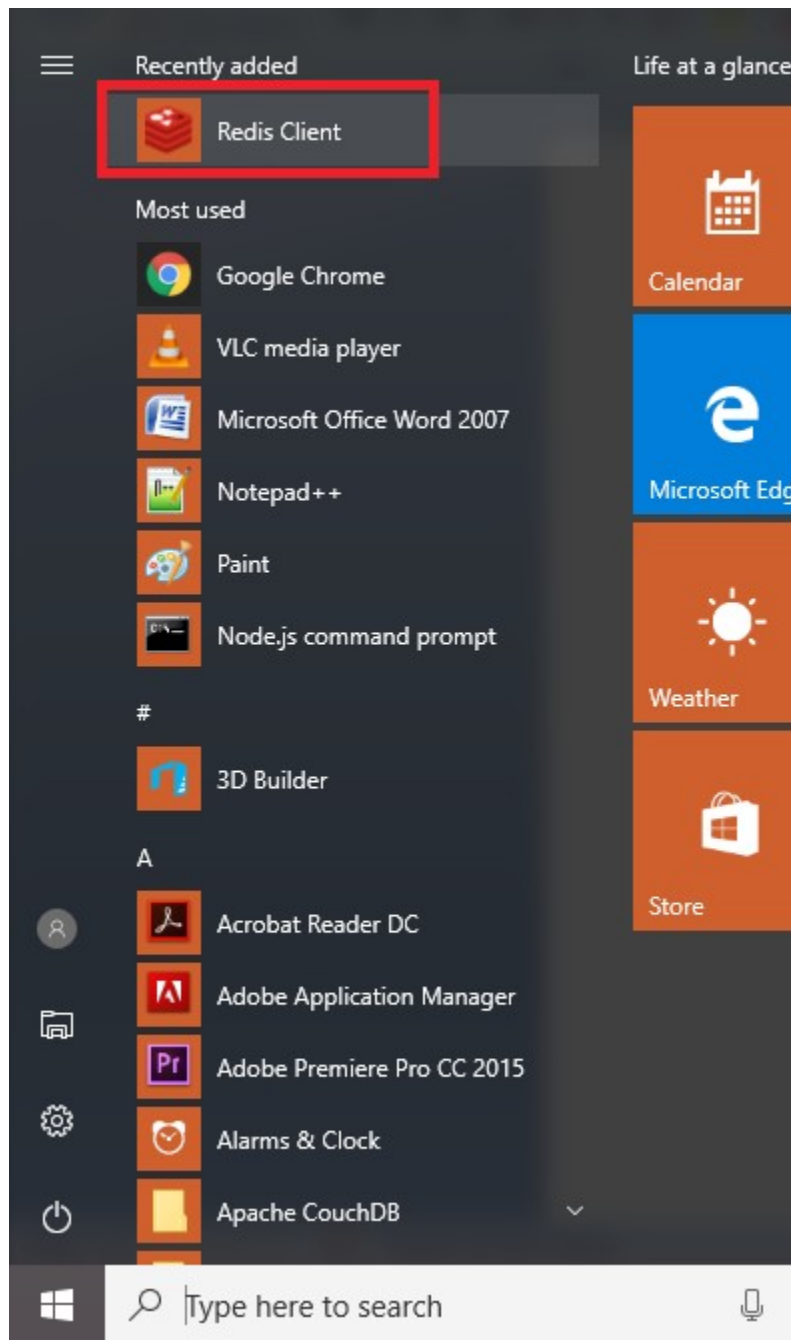
ZSCAN key cursor [MATCH pattern] [COUNT count]

Incrementally iterate sorted sets elements and associated scores

Next Topic [Redis Commands](#)

Redis Commands

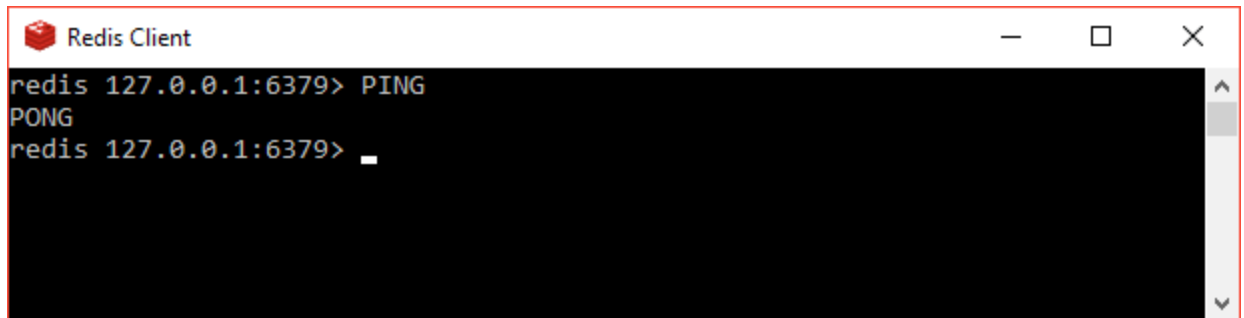
Redis commands are used for performing some operation on Redis Server. You must have a Redis client to run Redis command.



Start Redis Client

To start Redis client, first go to "Program files" where Redis is installed. Open the folder Redis and click on the "redis-server.exe".

Let's write a command "PING". If it returns a response "PONG" that means you are connected.

A screenshot of a window titled "Redis Client". The window has a black background with white text. The text shows a command prompt where the user has entered "redis 127.0.0.1:6379> PING" and the response "PONG" is displayed. The prompt "redis 127.0.0.1:6379>" is followed by a cursor. The window has standard Windows window controls (minimize, maximize, close) in the top right corner.

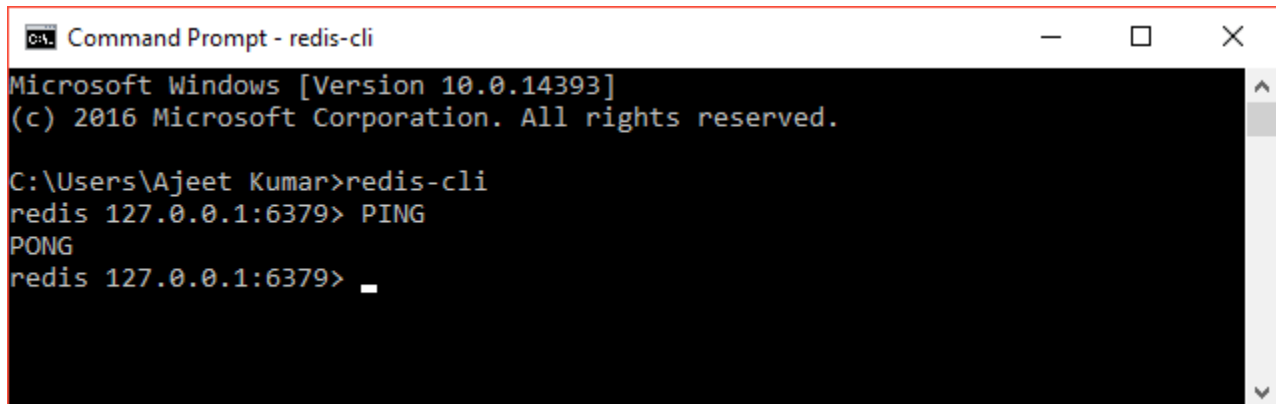
```
redis 127.0.0.1:6379> PING
PONG
redis 127.0.0.1:6379> _
```

Run Commands on the Remote Server

You can also connect to another server by using redis-cli command on windows command prompt:

Syntax

1. `$ redis-cli -h host -p port -a password`

A screenshot of a Windows Command Prompt window titled "Command Prompt - redis-cli". The window shows the output of the "redis-cli" command. It displays the Microsoft Windows version (10.0.14393) and copyright information. The user's current directory is "C:\Users\Ajeet Kumar". The user has entered "redis-cli" and then "redis 127.0.0.1:6379> PING", with the response "PONG" shown. The prompt "redis 127.0.0.1:6379>" is followed by a cursor. The window has standard Windows window controls in the top right corner.

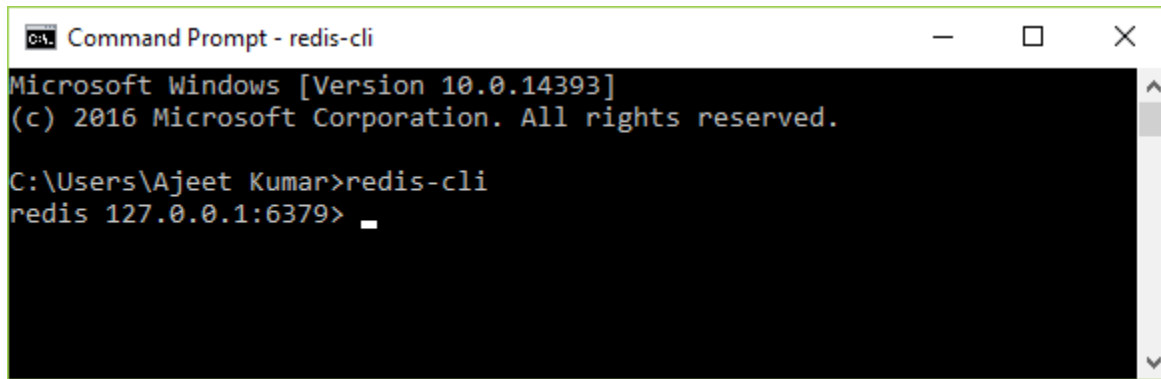
```
Microsoft Windows [Version 10.0.14393]
(c) 2016 Microsoft Corporation. All rights reserved.

C:\Users\Ajeet Kumar>redis-cli
redis 127.0.0.1:6379> PING
PONG
redis 127.0.0.1:6379> _
```

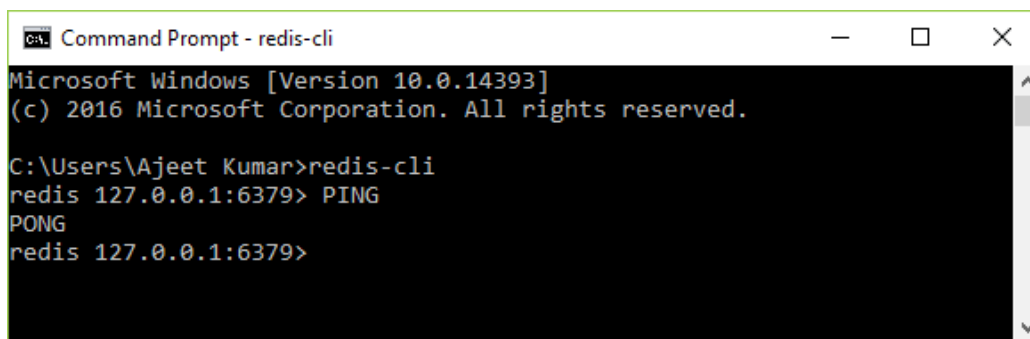
Example

Let's take an example to see how to connect to Redis remote server. Open command prompt and run the following command:

1. `redis-cli`



Now the Redis port is open. You can use the "PING" command to check if it is connected.



Now you can see that the remote server is connected to Redis.

Next Topic [Redis Keys](#)

Redis Keys

Redis Keys are used with Redis command to manage themselves. See the syntax of using Redis keys with commands.

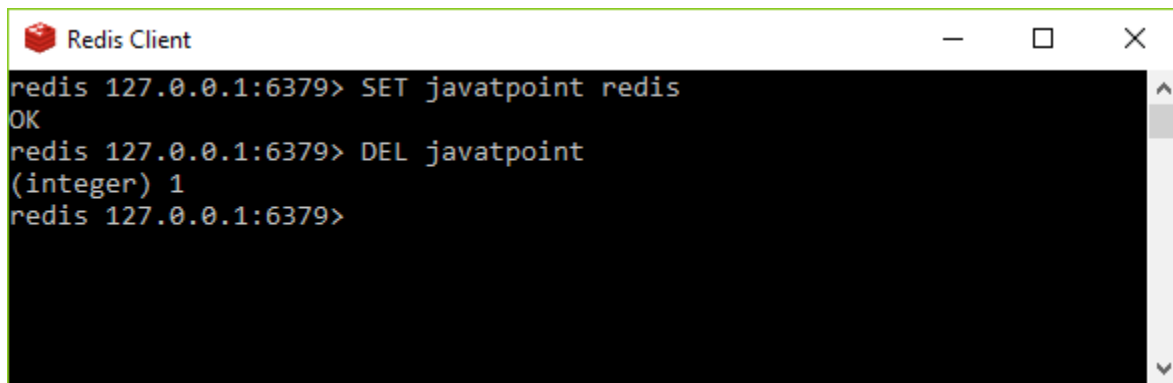
Syntax:

1. redis 127.0.0.1:6379> COMMAND KEY_NAME

Example

Let's take an example of Redis key using with Redis DEL command. It will give an output 1, if the key is deleted otherwise it will be 0.

1. redis 127.0.0.1:6379> SET javatpoint redis
2. OK
3. redis 127.0.0.1:6379> DEL javatpoint
4. (integer) 1

A screenshot of a Redis Client terminal window. The window has a title bar with the Redis logo and the text "Redis Client". The terminal shows the following commands and outputs:

```
redis 127.0.0.1:6379> SET javatpoint redis
OK
redis 127.0.0.1:6379> DEL javatpoint
(integer) 1
redis 127.0.0.1:6379>
```

Here, "DEL" is a Redis command while "javatpoint" is a key.

Redis Keys with Commands

Index	Command	Description
1	DEL key	This command is used to delete the key, if it exists.
2	DUMP key	This command is used to return a serialized version of the value stored at the key.
3	EXISTS key	This command is used to check whether the key exists or not.
4	EXPIRE key	This command is used to set the expiry of the key after the specified number of seconds.
5	EXPIREAT key	This command is used to set the expiry of the key after the specified UNIX timestamp in UNIX timestamp format.
6	PEXPIRE key	This command is used to set the expiry of key in milliseconds.
7	PEXPIREAT key	This command is used to set the expiry of the key in UNIX timestamp in milliseconds.
8	KEYS pattern	This command is used to find all keys matching the specified pattern.
9	MOVE key	It is used to move a key to another database.
10	PERSIST key	It is used to remove the expiration from the key.

11	PTTL key	It is used to retrieve the remaining time in keys expiry in milliseconds.
12	TTL key	It is used to retrieve the remaining time in keys expiry.
13	randomkey	It is used to get a random key from Redis.
14	RENAME key newkey	It is used to change the key name.
15	RENAMENX key newkey	It is used to rename the key, if a new key doesn't exist.
16	TYPE key	It is used to fetch the data type of the value stored in the key.

Next Topic [Redis Strings](#)

Redis String

Redis strings commands are used to manage string values in Redis. See the syntax for using Redis string commands.

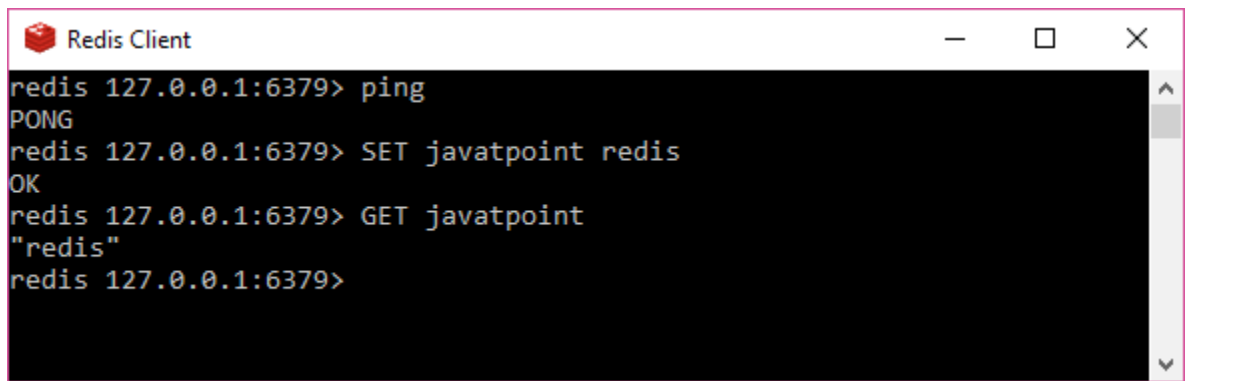
Syntax:

1. redis **127.0.0.1:6379**> COMMAND KEY_NAME

Example

1. redis **127.0.0.1:6379**> SET javatpoint redis
2. OK
3. redis **127.0.0.1:6379**> GET javatpoint
4. "redis"

Here, SET and GET are commands and "javatpoint" is key.

A screenshot of a terminal window titled "Redis Client". The window has a black background with white text. The text shows a series of commands and responses: "redis 127.0.0.1:6379> ping" followed by "PONG", "redis 127.0.0.1:6379> SET javatpoint redis" followed by "OK", "redis 127.0.0.1:6379> GET javatpoint" followed by "\"redis\"", and finally "redis 127.0.0.1:6379>". The window has standard macOS window controls (red, yellow, green buttons) in the top left corner and a scrollbar on the right side.

```
redis 127.0.0.1:6379> ping
PONG
redis 127.0.0.1:6379> SET javatpoint redis
OK
redis 127.0.0.1:6379> GET javatpoint
"redis"
redis 127.0.0.1:6379>
```

Redis Strings Commands

Following is a list of some basic commands which are used to manage strings in Redis.

Index	Command	Description
1	SET key value	This command is used to set the value at the specified key.
2	GET key	This command is used to retrieve the value of a key.
3	GETRANGE key start end	This command is used to get a substring of the string stored at the key.
4	GETSET key value	This command is used to set the string value of a key and return the old value.
5	GETBIT key offset	This command is used to return the bit value at the offset stored at the key.
6	MGET key1 [key2..]	This command is used to get the values of all the given keys.
7	SETBIT key offset value	This command is used to set or clear the bit at the offset stored at the key.
8	SETEX key seconds value	This command is used to set the value with the expiry of a key.
9	SETNX key value	This command is used to set the value of a key, only if the key does not exist.
10	SETRANGE key offset value	This command is used to overwrite the part of a string at the specified offset.

11	STRLEN key	This command is used to retrieve the length of the value s
12	MSET key value [key value ...]	This command is used to set multiple keys to multiple valu
13	MSETNX key value [key value ...]	This command is used to set multiple keys to multiple valu the keys exist
14	PSETEX key milliseconds value	This command is used to set the value and expiration in m
15	INCR key	This command is used to increment the integer value of a
16	INCRBY key increment	This command is used to increment the integer value of a amount
17	INCRBYFLOAT key increment	This command is used to increment the float value of a ke amount
18	DECR key	This command is used to decrement the integer value of a
19	DECRBY key decrement	This command is used to decrement the integer value of a number
20	APPEND key value	This command is used to append a value to a key

Next Topic [Redis Hashes](#)

Redis Hashes

Redis Hashes are the perfect data type to represent objects. They used to map between the string fields and the string values. In Redis, every hash can store up to more than 4 billion field-value pairs.

Example

```
redis 127.0.0.1:6379> HMSET javatpoint name "A solution to all Technology" description  
"India's fastest growing tutorial website" daily 1 million visitors 10 millions page visit.
```

1. OK
2. redis 127.0.0.1:6379> HGETALL javatpoint
3. 1) "name"
4. 2) "A solution to all Technology"
5. 3) "description"
6. 4) "India's fastest growing tutorial website"
7. 5) "daily"
8. 6) "1"

9. 7) "million"
10. 8) "visitors"
11. 9) "10"
12. 10) "millions"
13. 11) "page"
14. 12) "visit."
15. redis 127.0.0.1:6379>

```
Select Redis Client
redis 127.0.0.1:6379> set all Technology" description "India's fastest growing tutorial website" daily 1 million visitors 10 millions page visit.
OK
redis 127.0.0.1:6379> HGETALL javatpoint
1) "name"
2) "A solution to all Technology"
3) "description"
4) "India's fastest growing tutorial website"
5) "daily"
6) "1"
7) "million"
8) "visitors"
9) "10"
10) "millions"
11) "page"
12) "visit."
redis 127.0.0.1:6379>
```

```
Redis Client
redis 127.0.0.1:6379> ping
PONG
redis 127.0.0.1:6379> HGETALL javatpoint
(empty list or set)
redis 127.0.0.1:6379> set ion "India's fastest growing tutorial website" daily 1 million visitors 10 millions page visit.
OK
redis 127.0.0.1:6379> HGETALL javatpoint
1) "name"
2) "A solution to all Technology"
3) "description"
4) "India's fastest growing tutorial website"
5) "daily"
6) "1"
7) "million"
8) "visitors"
9) "10"
10) "millions"
11) "page"
12) "visit."
redis 127.0.0.1:6379>
```

In the above example, "javatpoint" is Redis hash which contains the details (name, description, 1, million, 10, visitors, page, visit) etc.

Redis Hash Commands

Index	Command	Description
1	HDEL key field2 [field2]	It is used to delete one or more hash fields.
2	HEXISTS key field	It is used to determine whether a hash field exists
3	HGET key field	It is used to get the value of a hash field stored at
4	HGETALL key	It is used to get all the fields and values stored in specified key
5	HINCRBY key field increment	It is used to increment the integer value of a hash number
6	HINCRBYFLOAT key field increment	It is used to increment the float value of a hash field by amount
7	HKEYS key	It is used to get all the fields in a hash
8	HLEN key	It is used to get the number of fields in a hash
9	HMGET key field1 [field2]	It is used to get the values of all the given hash fields
10	HMSET key field1 value1 [field2 value2]	It is used to set multiple hash fields to multiple values
11	HSET key field value	It is used to set the string value of a hash field
12	HSETNX key field value	It is used to set the value of a hash field, only if the field does not exist
13	HVALS key	It is used to get all the values in a hash

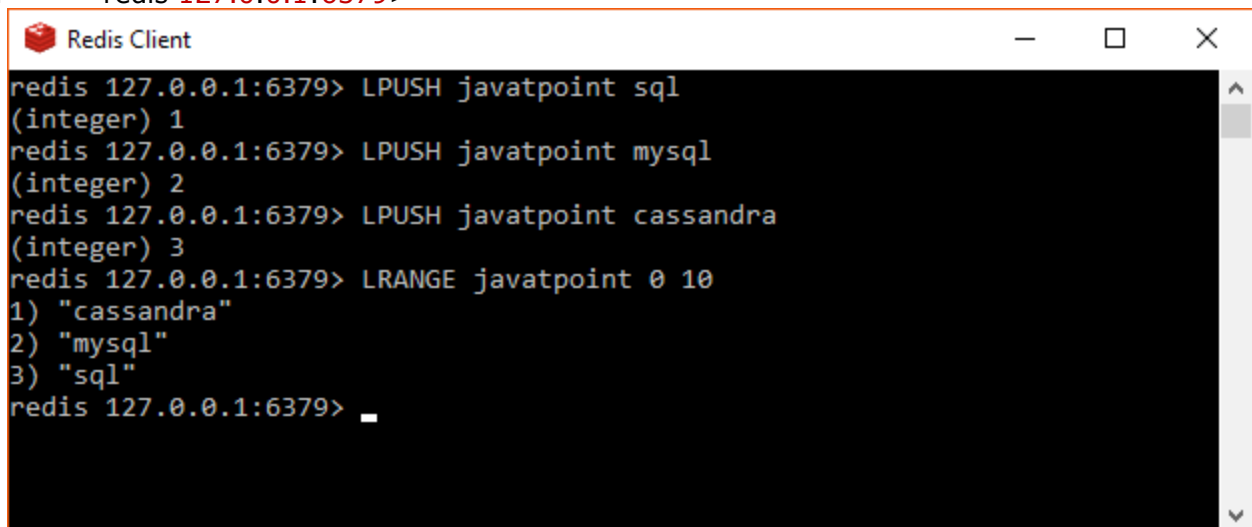
Redis Lists

Redis lists can be defined as a list of strings, sorted by insertion order. You can add elements in Redis lists in the head or the tail of the list.

A list can contain more than 4 billion elements.

Example

1. redis 127.0.0.1:6379> LPUSH javatpoint sql
2. (integer) 1
3. redis 127.0.0.1:6379> LPUSH javatpoint mysql
4. (integer) 2
5. redis 127.0.0.1:6379> LPUSH javatpoint cassandra
6. (integer) 3
7. redis 127.0.0.1:6379> LRANGE javatpoint 0 10
8. 1) "cassandra"
9. 2) "mysql"
10. 3) "sql"
11. redis 127.0.0.1:6379>

A screenshot of a terminal window titled "Redis Client". The window has a black background with white text. It shows the following commands and their outputs:

```
redis 127.0.0.1:6379> LPUSH javatpoint sql
(integer) 1
redis 127.0.0.1:6379> LPUSH javatpoint mysql
(integer) 2
redis 127.0.0.1:6379> LPUSH javatpoint cassandra
(integer) 3
redis 127.0.0.1:6379> LRANGE javatpoint 0 10
1) "cassandra"
2) "mysql"
3) "sql"
redis 127.0.0.1:6379> _
```

The terminal window has standard window controls (minimize, maximize, close) in the top right corner.

Redis Lists Commands

Index	Command	Description
1	BLPOP key1 [key2] timeout	It is used to remove and get the first element in a list, or blocks until one is available.
2	BRPOP key1 [key2] timeout	It is used to remove and get the last element in a list, or blocks until one is available.
3	BRPOPLPUSH source destination timeout	It is used to pop a value from a list, pushes it to another list, or blocks until one is available.
4	LINDEX key index	It is used to get an element from a list by its index.
5	LINSERT key before after pivot value	It is used to insert an element before or after another element in a list.
6	LLEN key	It is used to get the length of a list.
7	LPOP key	It is used to remove and get the first element in a list.
8	LPUSH key value1 [value2]	It is used to prepend one or multiple values to a list.
9	LPUSHX key value	It is used to prepend a value to a list, only if the list exists.
10	LRANGE key start stop	It is used to get a range of elements from a list.
11	LREM key count value	It is used to remove elements from a list.
12	LSET key index value	It is used to set the value of an element in a list by its index.

13	LTRIM key start stop	It is used to trim a list to the specified range.
14	RPOP key	It is used to remove and get the last element in a list.
15	RPOPLPUSH source destination	It is used to remove the last element in a list, append it to the destination list, and returns it.
16	R PUSH key value1 [value2]	It is used to append one or multiple values to a list.
17	R PUSHX key value	It is used to append a value to a list, only if the list exists.

Next Topic [Redis Sets](#)

Redis Sets

Redis Sets are an unordered collection of unique strings. Unique strings means there is not a single string repeated in the set.

In Redis set add, remove, and test for the existence of members in $O(1)$ (constant time regardless of the number of elements contained inside the Set). The maximum length of a list is more than 4 billion of elements per set.

EXAMPLE

1. redis 127.0.0.1:6379> SADD javatpoint db2
2. (integer) 1
3. redis 127.0.0.1:6379> SADD javatpoint mongodb
4. (integer) 1
5. redis 127.0.0.1:6379> SADD javatpoint db2
6. (integer) 0
7. redis 127.0.0.1:6379> SADD javatpoint cassandra
8. (integer) 1
9. redis 127.0.0.1:6379> SMEMBERS javatpoint
10. 1) "cassandra"
11. 2) "db2"
12. 3) "mongodb"

```
Redis Client
redis 127.0.0.1:6379> SADD javatpoint db2
(integer) 1
redis 127.0.0.1:6379> SADD javatpoint mongodb
(integer) 1
redis 127.0.0.1:6379> SADD javatpoint db2
(integer) 0
redis 127.0.0.1:6379> SADD javatpoint cassandra
(integer) 1
redis 127.0.0.1:6379> SMEMBERS javatpoint
1) "cassandra"
2) "db2"
3) "mongodb"
redis 127.0.0.1:6379> _
```

In the above example, you can see that we have added 4 elements in the set by using SADD command. But only 3 elements are retrieved by using SMEMBERS command because one element was duplicate and Redis sets read duplicate values only once.

Redis Sets Commands

Index	Command	Description
1	SADD key member1 [member2]	It is used to add one or more members to a
2	SCARD key	It is used to get the number of members in
3	SDIFF key1 [key2]	It is used to subtract multiple sets.
4	SDIFFSTORE destination key1 [key2]	It is used to subtract multiple sets and store the result in a key.
5	SINTER key1 [key2]	It is used to intersect multiple sets.
6	SINTERSTORE destination key1 [key2]	It is used to intersect multiple sets and store the result in a key.
7	SISMEMBER key member	It is used to determine if a given value is a

8	SMOVE source destination member	It is used to move a member from one set
9	SPOP key	It is used to remove and returns a random
10	SRANDMEMBER key [count]	It is used to get one or multiple random me
11	SREM key member1 [member2]	It is used to remove one or more members
12	SUNION key1 [key2]	It is used to add multiple sets.
13	SUNIONSTORE destination key1 [key2]	It is used to add multiple sets and stores th key.
14	SSCAN key cursor [match pattern] [count count]	It is used to incrementally iterates set elem

Next Topic [Redis Sorted Sets](#)

Redis Sorted Sets

Redis Sorted Sets are similar to Redis Sets but the first one has the unique feature of values stored. It means every member of a Sorted Set is associated with a score which can be used to take the sorted set ordered, from the smallest to the greatest score.

Example

1. redis 127.0.0.1:6379> ZADD javatpoint 1 redis
2. (integer) 0
3. redis 127.0.0.1:6379> ZADD javatpoint 2 cassandra
4. (integer) 1
5. redis 127.0.0.1:6379> ZADD javatpoint 3 cassandra
6. (integer) 0
7. redis 127.0.0.1:6379> ZADD javatpoint 3 mysql
8. (integer) 1
9. redis 127.0.0.1:6379> ZADD javatpoint 4 mysql
10. (integer) 0
11. redis 127.0.0.1:6379> ZRANGE javatpoint 0 10 WITHSCORES
12. 1) "redis"
13. 2) "1"
14. 3) "cassandra"

15. 4) "3"
16. 5) "mysql"
17. 6) "4"

```
Redis Client
redis 127.0.0.1:6379> ZADD javatpoint 1 redis
(integer) 0
redis 127.0.0.1:6379> ZADD javatpoint 2 cassandra
(integer) 1
redis 127.0.0.1:6379> ZADD javatpoint 3 cassandra
(integer) 0
redis 127.0.0.1:6379> ZADD javatpoint 3 mysql
(integer) 1
redis 127.0.0.1:6379> ZADD javatpoint 4 mysql
(integer) 0
redis 127.0.0.1:6379> ZRANGE javatpoint 0 10 WITHSCORES
1) "redis"
2) "1"
3) "cassandra"
4) "3"
5) "mysql"
6) "4"
redis 127.0.0.1:6379> _
```

Redis Sorted Sets Commands

Following is a list of commands used in sorted sets.

Index	Commands	Description
1	ZADD key score1 member1 [score2 member2]	It is used to add one or more members to a sorted set, or update its score, if it already exists
2	ZCARD key	It is used to get the number of members in a sorted set
3	ZCOUNT key min max	It is used to count the members in a sorted set by their scores within the given values
4	ZINCRBY key increment member	It is used to increment the score of a member in a sorted set

5	ZINTERSTORE destination numkeys key [key ...]	It is used to intersect multiple sorted sets and store the result in a new key
6	ZLEXCOUNT key min max	It is used to count the number of members in a sorted set between a given lexicographical range
7	ZRANGE key start stop [WITHSCORES]	It is used to return a range of members in a sorted set
8	ZRANGEBYLEX key min max [LIMIT offset count]	It is used to return a range of members in a sorted set between a given lexicographical range
9	ZRANGEBYSCORE key min max [WITHSCORES] [LIMIT]	It is used to return a range of members in a sorted set between a given score range
10	ZRANK key member	It is used to determine the index of a member in a sorted set
11	ZREM key member [member ...]	It is used to remove one or more members from a sorted set
12	ZREMRANGEBYLEX key min max	It is used to remove all members in a sorted set between a given lexicographical range
13	ZREMRANGEBYRANK key start stop	It is used to remove all members in a sorted set between a given index range
14	ZREMRANGEBYSCORE key min max	It is used to remove all members in a sorted set between a given score range
15	ZREVRANGE key start stop [WITHSCORES]	It is used to return a range of members in a sorted set with scores ordered from high to low
16	ZREVRANGEBYSCORE key max min [WITHSCORES]	It is used to return a range of members in a sorted set between a given score range with scores ordered from high to low
17	ZREVRANK key member	It is used to determine the index of a member in a sorted set with scores ordered from high to low
18	ZSCORE key member	It is used to retrieve the score associated with a member in a sorted set
19	ZUNIONSTORE destination numkeys key	It is used to add multiple sorted sets and store the result in a new key

	[key ...]	sorted set in a new key
20	ZSCAN key cursor [MATCH pattern] [COUNT count]	It is used to incrementally iterates sorted sets associated scores

Next Topic [Redis Transaction](#)

Redis Transactions

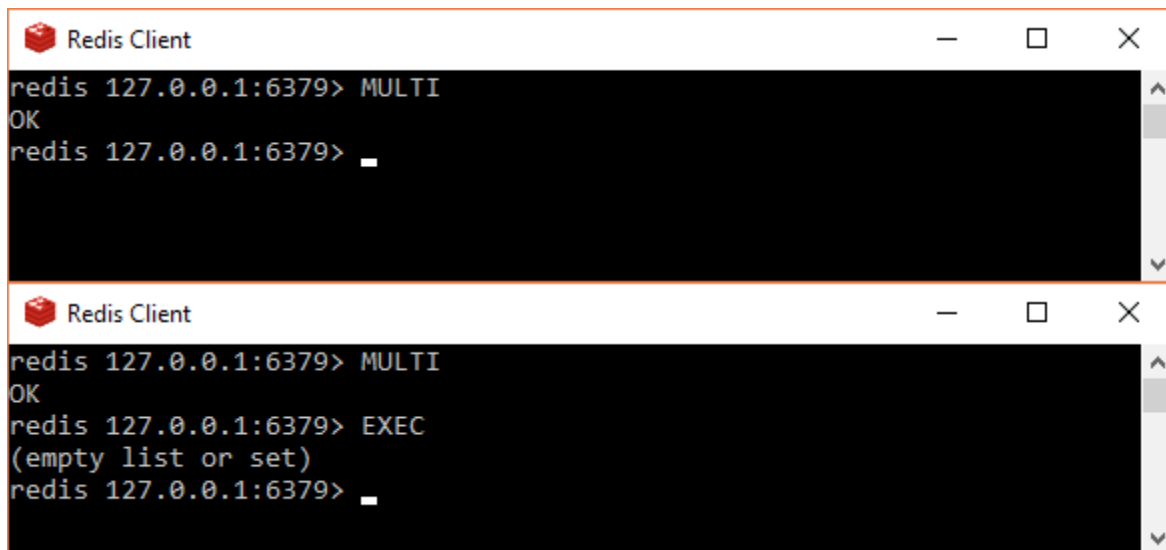
Redis transaction is used to facilitates users to execute group of commands in a single step.

There are two properties of execution:

- All commands in a transaction are sequentially executed as a single isolated operation. You can't issue a request by another client served in the middle of the execution of a Redis transaction.
- Redis transaction is also atomic. Atomic means either all of the commands or none are processed.

Sample

In Redis, transaction is initiated by using "MULTI" command and then you need to pass a list of commands that should be executed in the transaction, after which the entire transaction is executed by "EXEC" command.



The image shows two screenshots of the Redis Client terminal window. The top screenshot shows the command 'MULTI' being entered, followed by 'OK' and a cursor. The bottom screenshot shows the command 'MULTI' being entered, followed by 'OK', then 'EXEC' being entered, followed by '(empty list or set)' and a cursor.

```
redis 127.0.0.1:6379> MULTI
OK
redis 127.0.0.1:6379> _
```

```
redis 127.0.0.1:6379> MULTI
OK
redis 127.0.0.1:6379> EXEC
(empty list or set)
redis 127.0.0.1:6379> _
```

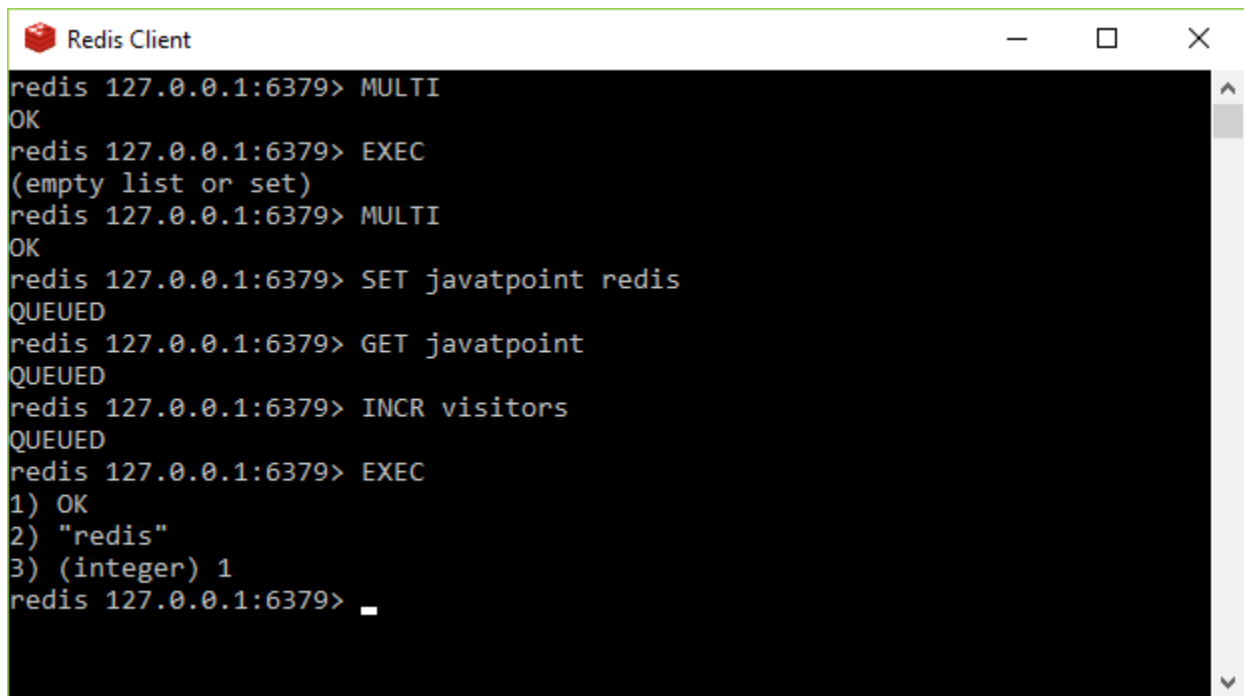
Example

Let's take an example to see how Redis transaction can be initiated and executed.

Example

1. redis 127.0.0.1:6379> MULTI
2. OK
3. redis 127.0.0.1:6379> EXEC

4. (empty list or set)
5. redis 127.0.0.1:6379> MULTI
6. OK
7. redis 127.0.0.1:6379> SET javatpoint redis
8. QUEUED
9. redis 127.0.0.1:6379> GET javatpoint
10. QUEUED
11. redis 127.0.0.1:6379> INCR visitors
12. QUEUED
13. redis 127.0.0.1:6379> EXEC
14. 1) OK
15. 2) "redis"
16. 3) (integer) 1



```
Redis Client
redis 127.0.0.1:6379> MULTI
OK
redis 127.0.0.1:6379> EXEC
(empty list or set)
redis 127.0.0.1:6379> MULTI
OK
redis 127.0.0.1:6379> SET javatpoint redis
QUEUED
redis 127.0.0.1:6379> GET javatpoint
QUEUED
redis 127.0.0.1:6379> INCR visitors
QUEUED
redis 127.0.0.1:6379> EXEC
1) OK
2) "redis"
3) (integer) 1
redis 127.0.0.1:6379> _
```

Redis Transaction Commands

Following is a list of some basic commands of Redis transaction.

Index	Command	Description
1	DISCARD	It is used to discard all commands issued after MULTI

2	EXEC	It is used to execute all commands issued after MULTI
3	MULTI	It is used to mark the start of a transaction block
4	UNWATCH	It is used to forget about all watched keys
5	WATCH key [key ...]	It is used to watch the given keys to determine the execution of the block

Next Topic [Redis Scripting](#)

Redis Scripting

Redis scripting is used to evaluate scripts using the Lua interpreter. It is built into Redis starting from version 2.6.0. The command used for scripting is EVAL command.

See the syntax of EVAL command.

Syntax

1. redis 127.0.0.1:6379> EVAL script numkeys key [key ...] arg [arg ...]

Example

Let's take an example to see how Redis scripting works:

1. redis 127.0.0.1:6379> EVAL "return {KEYS[1],KEYS[2],ARGV[1],ARGV[2]}" 2 key1
2. key2 first second
3. 1) "key1"
4. 2) "key2"
5. 3) "first"
6. 4) "second"

Next Topic [Redis Connection](#)

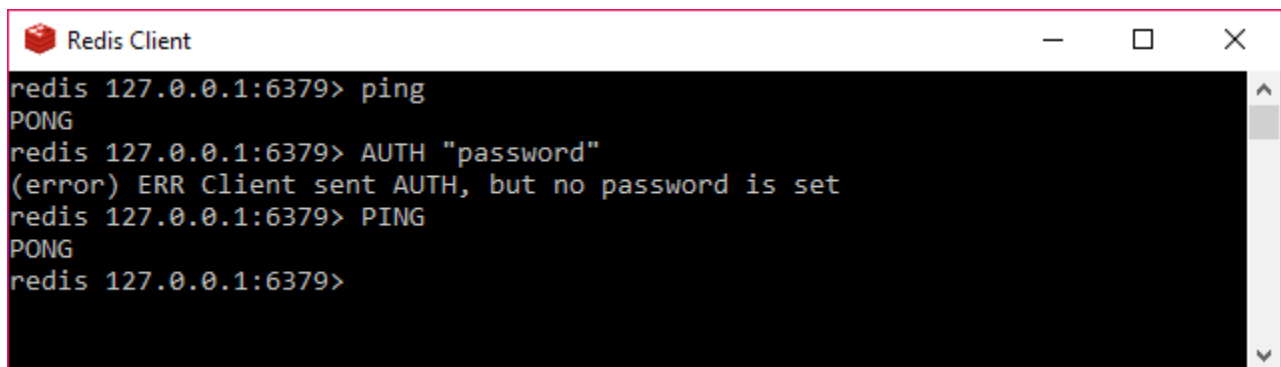
Redis Connections

Redis connection commands are used to control and manage client connections with Redis Server.

Example

Following example explains how a client authenticates itself to Redis server and checks whether the server is running or not.

1. redis 127.0.0.1:6379> AUTH "password"
2. (error) ERR Client sent AUTH, but no password is set
3. redis 127.0.0.1:6379>
4. redis 127.0.0.1:6379> PING
5. PONG
6. redis 127.0.0.1:6379>



```
Redis Client
redis 127.0.0.1:6379> ping
PONG
redis 127.0.0.1:6379> AUTH "password"
(error) ERR Client sent AUTH, but no password is set
redis 127.0.0.1:6379> PING
PONG
redis 127.0.0.1:6379>
```

Note: Here you can see that "password" is not set so you can access any command directly.

Redis Connection Commands

Following is a list of some basic connection commands used in Redis database:

Index	Commands	Description
1	AUTH password	It is used to authenticate to the server with the given password.

2	ECHO message	It is used to print the given string.
3	PING	It is used to check whether the server is running or not.
4	QUIT	It is used to close the current connection.
5	SELECT index	It is used to change the selected database for the current connection.

Next Topic [Redis Server](#)

Redis Server

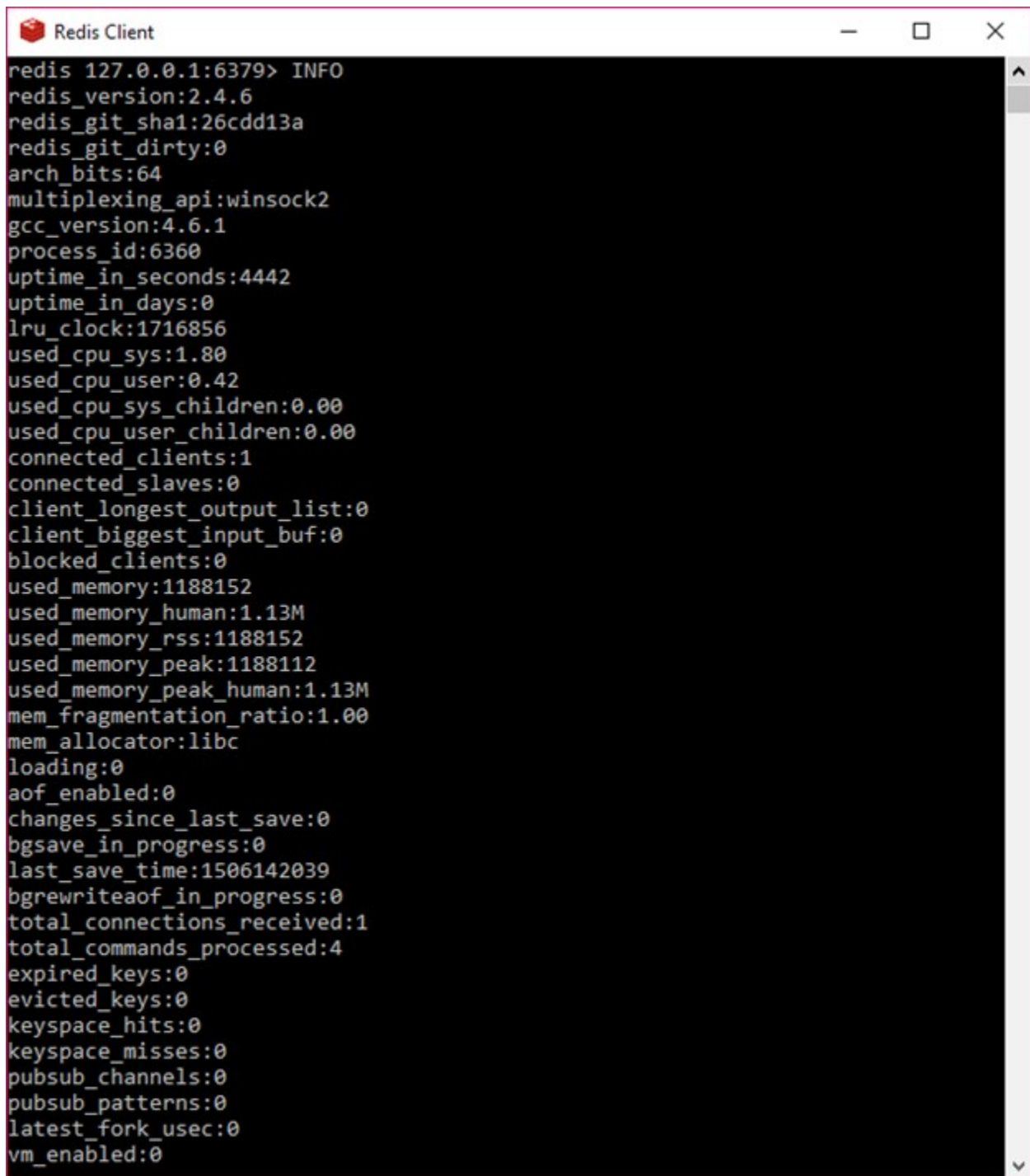
Redis Server commands are used to manage Redis Server. There are different server commands which can be used to fetch server information, statistics and other characteristics of server.

Example

Let's take an example to see how to get all statistics and information about the server.

```
1.      redis 127.0.0.1:6379> ping
2.      PONG
3.      redis 127.0.0.1:6379> AUTH "password"
4.      (error) ERR Client sent AUTH, but no password is set
5.      redis 127.0.0.1:6379> PING
6.      PONG
7.      redis 127.0.0.1:6379> ECHO "Welcome to Javatpoint"
8.      "Welcome to Javatpoint"
9.      redis 127.0.0.1:6379> INFO
10.     redis_version:2.4.6
11.     redis_git_sha1:26cdd13a
12.     redis_git_dirty:0
13.     arch_bits:64
14.     multiplexing_api:winsock2
15.     gcc_version:4.6.1
16.     process_id:6360
17.     uptime_in_seconds:4442
18.     uptime_in_days:0
19.     lru_clock:1716856
20.     used_cpu_sys:1.80
21.     used_cpu_user:0.42
22.     used_cpu_sys_children:0.00
23.     used_cpu_user_children:0.00
24.     connected_clients:1
25.     connected_slaves:0
26.     client_longest_output_list:0
27.     client_biggest_input_buf:0
28.     blocked_clients:0
29.     used_memory:1188152
30.     used_memory_human:1.13M
```

31. used_memory_rss:1188152
32. used_memory_peak:1188112
33. used_memory_peak_human:1.13M
34. mem_fragmentation_ratio:1.00
35. mem_allocator:libc
36. loading:0
37. aof_enabled:0
38. changes_since_last_save:0
39. bgsave_in_progress:0
40. last_save_time:1506142039
41. bgrewriteaof_in_progress:0
42. total_connections_received:1
43. total_commands_processed:4
44. expired_keys:0
45. evicted_keys:0
46. keyspace_hits:0
47. keyspace_misses:0
48. pubsub_channels:0
49. pubsub_patterns:0
50. latest_fork_usec:0
51. vm_enabled:0
52. role:master

A screenshot of a terminal window titled "Redis Client". The window has a standard macOS-style title bar with a red close button, a yellow maximize button, and a green window control button. The terminal background is black with white text. The text shows the command "redis 127.0.0.1:6379> INFO" followed by a list of Redis server statistics and configuration details. The output includes version information, memory usage, uptime, and various performance metrics.

```
redis 127.0.0.1:6379> INFO
redis_version:2.4.6
redis_git_sha1:26cdd13a
redis_git_dirty:0
arch_bits:64
multiplexing_api:winsock2
gcc_version:4.6.1
process_id:6360
uptime_in_seconds:4442
uptime_in_days:0
lru_clock:1716856
used_cpu_sys:1.80
used_cpu_user:0.42
used_cpu_sys_children:0.00
used_cpu_user_children:0.00
connected_clients:1
connected_slaves:0
client_longest_output_list:0
client_biggest_input_buf:0
blocked_clients:0
used_memory:1188152
used_memory_human:1.13M
used_memory_rss:1188152
used_memory_peak:1188112
used_memory_peak_human:1.13M
mem_fragmentation_ratio:1.00
mem_allocator:libc
loading:0
aof_enabled:0
changes_since_last_save:0
bgsave_in_progress:0
last_save_time:1506142039
bgrewriteaof_in_progress:0
total_connections_received:1
total_commands_processed:4
expired_keys:0
evicted_keys:0
keyspace_hits:0
keyspace_misses:0
pubsub_channels:0
pubsub_patterns:0
latest_fork_usec:0
vm_enabled:0
```

Redis Server Commands

Following is a list of some server commands used in Redis database:

Index	Command	Description
1	BGREWRITEAOF	This command is used to asynchronously rewrite the file.
2	BGSAVE	This command is used to asynchronously save the data to disk.
3	CLIENT KILL [ip:port] [ID client-id]	This command is used to kill the connection of a client.
4	CLIENT LIST	This command is used to get the list of client connections to the server.
5	CLIENT GETNAME	This command is used to get the name of the current client.
6	CLIENT PAUSE timeout	This command is used to stop processing commands from clients for a specified time.
7	CLIENT SETNAME connection-name	This command is used to set the current client's connection name.
8	CLUSTER SLOTS	This command is used to get an array of Cluster Slot mappings.
9	COMMAND	This command is used to get an array of Redis command names.
10	COMMAND COUNT	This command is used to get total number of Redis commands.
11	COMMAND GETKEYS	This command is used to extract the keys given to the command.
12	BGSAVE	This command is used to asynchronously save the data to disk.
13	COMMAND INFO command-name [command-name ...]	This command is used to get an array of specific command details.
14	CONFIG GET parameter	This command is used to get the value of a configuration parameter.
15	CONFIG REWRITE	This command is used to rewrite the configuration file.

		memory configuration.
16	CONFIG SET parameter value	This command is used to get a configuration parameter value.
17	CONFIG RESETSTAT	This command is used to reset the stats returned by CONFIG GET.
18	DBSIZE	This command is used to return the number of keys in the database.
19	DEBUG OBJECT key	This command is used to get debugging information about a key.
20	DEBUG SEGFAULT	This command is used to make the server crash.
21	FLUSHALL	This command is used to remove all the keys from the database.
22	FLUSHDB	This command is used to remove all the keys from the database.
23	INFO [section]	This command is used to get information and statistics about the server.
24	LASTSAVE	This command is used to retrieve the UNIX time of the last successful save to the disk.
25	MONITOR	This command is used to listen for all the requests received by the server in real time.
26	ROLE	This command is used to return the role of the instance in the context of replication.
27	SAVE	This command is used to synchronously save the dataset to the disk.
28	SHUTDOWN [NOSAVE] [SAVE]	This command is used to synchronously save the dataset to the disk and then shuts down the server.
29	SLAVEOF host port	This command is used to make the server a slave of another Redis instance, or promotes it as a master.

30	SLOWLOG subcommand [argument]	This command is used to manage the Redis slow
31	SYNC	This command is used for replication.
32	TIME	This command is used to return the current serv

Next Topic [Redis Backup and Restore](#)

Redis Backup & Restore

SAVE command is used to create a backup of a current Redis database. This command will create a dump.rdb file in your Redis directory by performing synchronous save.

Syntax

1. SAVE

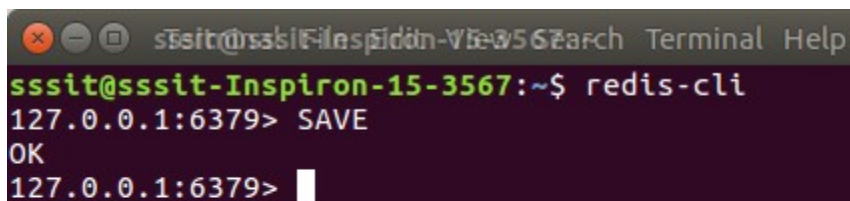
Return Value

The SAVE command returns OK after successful execution.

Redis Backup Example

Use the SAVE command to create a backup of the current database.

SAVE



```
sssit@sssit-Inspiron-15-3567:~$ redis-cli
127.0.0.1:6379> SAVE
OK
127.0.0.1:6379> 
```

It will create a dump.rdb file in your Redis directory.

You can see that the dump.rdb file is created.

```
sssit@sssit-Inspiron-15-3567: ~$ ls
Desktop    Downloads  examples.desktop  Pictures  Rashmi.xlsx  Videos
Documents  dump.rdb   Music             Public   Templates
```

Restore Redis Data

You have to move Redis backup file (dump.rdb) into your Redis directory and start the server to restore Redis data.

Use CONFIG command of Redis as shown below.

```
sssit@sssit-Inspiron-15-3567: ~$ redis-cli
127.0.0.1:6379> SAVE
OK
127.0.0.1:6379>
sssit@sssit-Inspiron-15-3567: ~$ redis-cli
127.0.0.1:6379> CONFIG get dir
1) "dir"
2) "/var/lib/redis"
127.0.0.1:6379>
```

The Redis server is installed in the following directory.

`"/var/lib/redis"`

BGSAVE Command

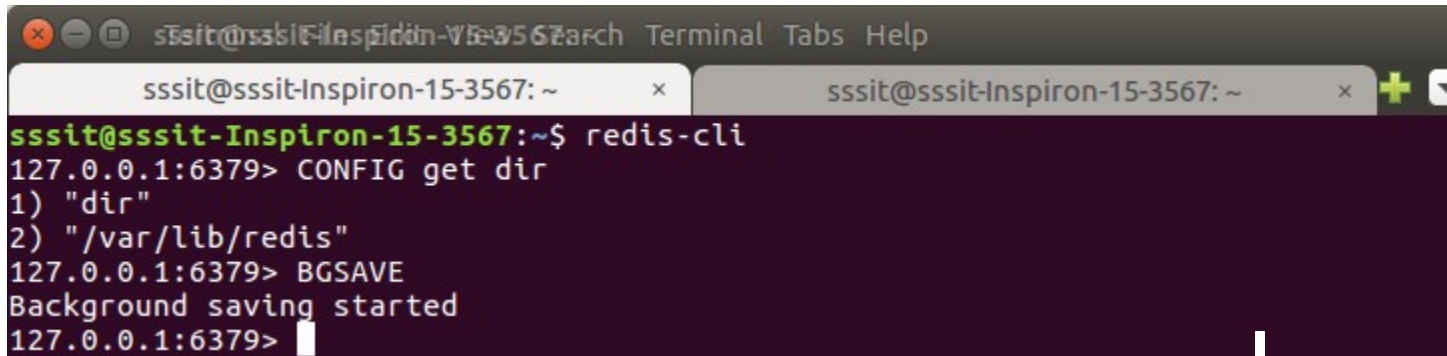
There is an alternate command named BGSAVE which is used to create Redis backup.

This command will start the backup process and run this in the background.

Syntax

1. BGSAVE

Example

A terminal window titled 'Terminal' with tabs for 'Terminal' and 'Help'. The terminal shows a session with the Redis CLI. The prompt is 'sssit@sssit-Inspiron-15-3567: ~'. The user enters 'redis-cli'. The prompt changes to '127.0.0.1:6379>'. The user enters 'CONFIG get dir'. The output is '1) "dir"' and '2) "/var/lib/redis"'. The user enters 'BGSAVE'. The output is 'Background saving started'. The prompt returns to '127.0.0.1:6379>'.

```
sssit@sssit-Inspiron-15-3567: ~$ redis-cli
127.0.0.1:6379> CONFIG get dir
1) "dir"
2) "/var/lib/redis"
127.0.0.1:6379> BGSAVE
Background saving started
127.0.0.1:6379>
```

Next Topic [Redis Security](#)

Redis Security

Security is very necessary for a database to keep the data safe and secure. It provides an authentication so if a client want to make a connection needs to authenticate before executing a command.

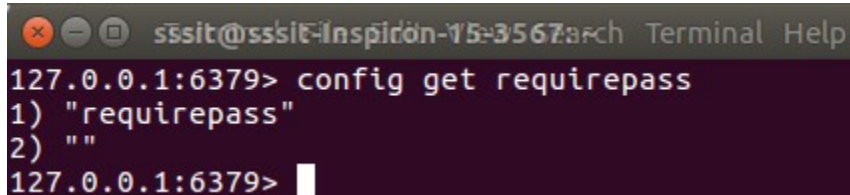
You need to set a password in the config file to secure a Redis database.

Example

Let's see how to secure your Redis instance.

Use the "config get command"

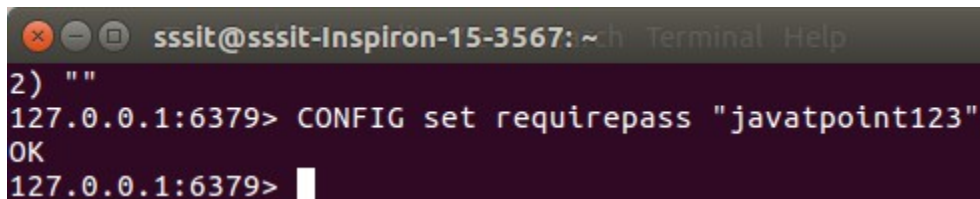
1. `config get requirepass`



```
sssit@sssit-Inspiron-15-3567: ~$ ssh Terminal Help
127.0.0.1:6379> config get requirepass
1) "requirepass"
2) ""
127.0.0.1:6379> █
```

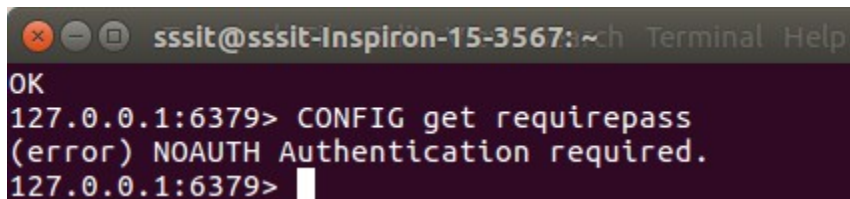
You can see that the above property is blank means we don't have any password for this instance. You can change this property and set a password for this instance by executing the following command.

1. `config set requirepass "javatpoint123"`



```
sssit@sssit-Inspiron-15-3567: ~$ ssh Terminal Help
2) ""
127.0.0.1:6379> CONFIG set requirepass "javatpoint123"
OK
127.0.0.1:6379> █
```

1. `CONFIG get requirepass`

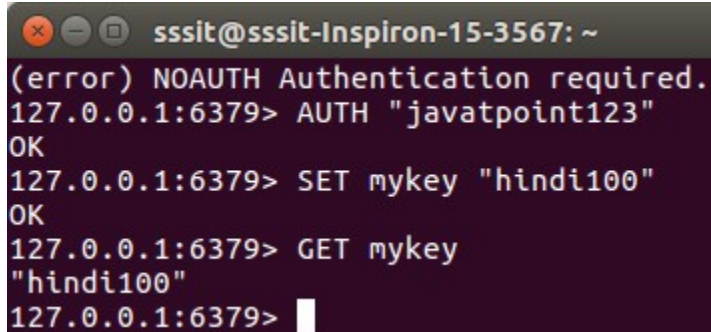


```
sssit@sssit-Inspiron-15-3567: ~$ ssh Terminal Help
OK
127.0.0.1:6379> CONFIG get requirepass
(error) NOAUTH Authentication required.
127.0.0.1:6379> █
```

When you set this password then if a client runs the command without authentication, he will get an error "NOAUTH Authentication required.". Hence, the client needs to use AUTH command to authenticate himself.

Usage of AUTH Command

1. `127.0.0.1:6379> AUTH "javatpoint123"`
2. `OK`
3. `127.0.0.1:6379> SET mykey "hindi100"`
4. `OK`
5. `127.0.0.1:6379> GET mykey`
6. `"hindi100"`
7. `127.0.0.1:6379>`



A terminal window titled 'sssit@sssit-Inspiron-15-3567: ~' showing the execution of Redis commands. The output shows an error for the first AUTH command, followed by successful responses for the subsequent SET and GET commands.

```
(error) NOAUTH Authentication required.  
127.0.0.1:6379> AUTH "javatpoint123"  
OK  
127.0.0.1:6379> SET mykey "hindi100"  
OK  
127.0.0.1:6379> GET mykey  
"hindi100"  
127.0.0.1:6379>
```

[Next Topic](#) [Redis Benchmarks](#)

Redis Benchmarks

Redis benchmark is a utility tool which is used to check the performance of Redis by using multiple (n) command simultaneously.

Syntax

1. `redis-benchmark [option] [option value]`

Example

Let's call Redis Benchmark commands:

```
redis-benchmark -n 100000
```

```
sssit@sssit-Inspiron-15-3567: ~  
sssit@sssit-Inspiron-15-3567:~$ redis-benchmark -n 100000  
===== PING_INLINE =====  
100000 requests completed in 0.75 seconds  
50 parallel clients  
3 bytes payload  
keep alive: 1  
  
99.89% <= 1 milliseconds  
100.00% <= 1 milliseconds  
132802.12 requests per second  
  
===== PING_BULK =====  
100000 requests completed in 0.68 seconds  
50 parallel clients  
3 bytes payload  
keep alive: 1  
  
99.99% <= 1 milliseconds  
100.00% <= 1 milliseconds  
147058.83 requests per second  
  
===== SET =====  
100000 requests completed in 0.77 seconds  
50 parallel clients
```

```
st@st01:~/redis-benchmark$ ./redis-benchmark -h 127.0.0.1 -p 6379 -c 50 -n 100000 -d 3 -k 1
100000 requests completed in 11.67 seconds
50 parallel clients
3 bytes payload
keep alive: 1

0.11% <= 1 milliseconds
6.18% <= 2 milliseconds
59.94% <= 3 milliseconds
94.49% <= 4 milliseconds
98.58% <= 5 milliseconds
99.46% <= 6 milliseconds
99.69% <= 7 milliseconds
99.81% <= 8 milliseconds
99.87% <= 9 milliseconds
99.90% <= 10 milliseconds
99.93% <= 11 milliseconds
99.94% <= 12 milliseconds
99.95% <= 13 milliseconds
99.96% <= 14 milliseconds
99.98% <= 15 milliseconds
99.99% <= 16 milliseconds
100.00% <= 16 milliseconds
8569.71 requests per second
```

See the list of available options in Redis benchmark:

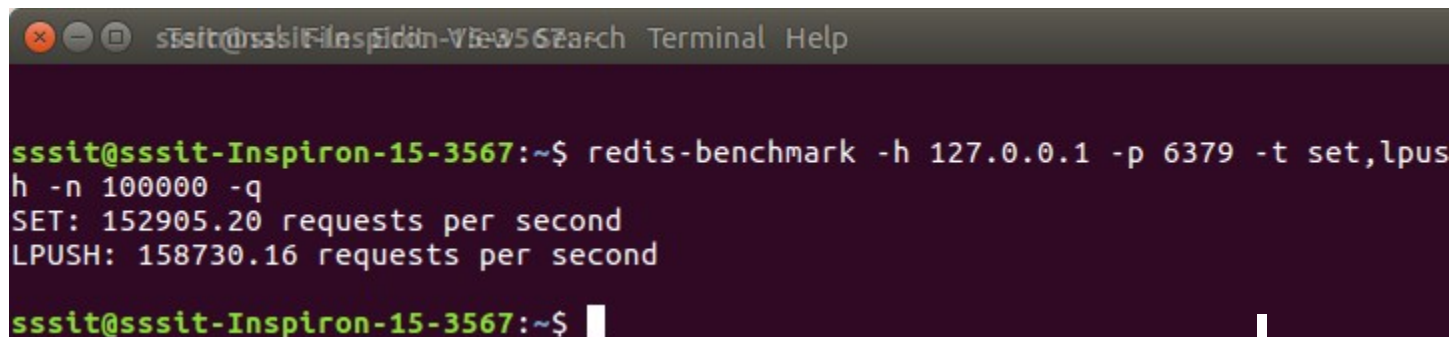
Index	Option	Description	
1	-h	Specifies server host name	
2	-p	Specifies server port	
3	-s	Specifies server socket	
4	-c	Specifies the number of parallel connections	
5	-n	Specifies the total number of requests	
6	-d	Specifies data size of SET/GET value in bytes	
7	-k	1=keep alive, 0=reconnect	
8	-r	Use random keys for SET/GET/INCR, random values for SADD	

9	-p	Pipeline requests	
10	-h	Specifies server host name	
11	-q	Forces Quiet to Redis. Just shows query/sec values	
12	--csv	Output in CSV format	
13	-l	Generates loop, Run the tests forever	
14	-t	Only runs the comma-separated list of tests	
15	-I	Idle mode. Just opens N idle connections and wait	

Example

Let's take an example to show multiple usage options in Redis benchmark utility.

1. `redis-benchmark -h 127.0.0.1 -p 6379 -t set,lpush -n 100000 -q`



```

sssit@sssit-Inspiron-15-3567:~$ redis-benchmark -h 127.0.0.1 -p 6379 -t set,lpush
h -n 100000 -q
SET: 152905.20 requests per second
LPUSH: 158730.16 requests per second
sssit@sssit-Inspiron-15-3567:~$

```

Next Topic [Redis Client Connection](#)

edis Client Connection

Redis can accept different type of clients' connections on the configured listening TCP port and on the Unix socket, if enabled.

When a new client connection is accepted, it performs the following operations:

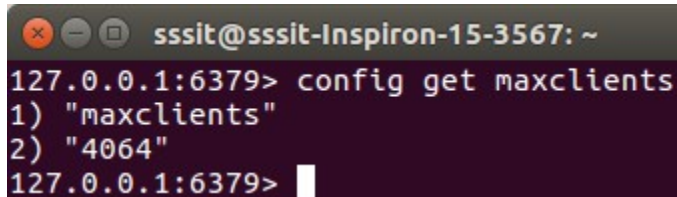
- The client socket is put in non-blocking state since Redis uses multiplexing and non-blocking I/O.
 - The TCP_NODELAY option is set in order to ensure that we don't have delays in our connection.
 - A readable file event is created so that Redis is able to collect the client queries as soon as new data is available to be read on the socket.
-

Maximum number of Clients

In Redis config (redis.conf), there is a property called maxclients, which specifies that how many number of clients can be connected to Redis.

Following is the basic syntax of command.

1. Config get maxclients
2. "maxclients"
3. "4064"



```
ssslit@ssslit-Inspiron-15-3567: ~  
127.0.0.1:6379> config get maxclients  
1) "maxclients"  
2) "4064"  
127.0.0.1:6379> 
```

The maximum number of clients depends upon the maximum number of file descriptors limit of OS. Its default value is 10000, although you can change this property.

Example

Let's take an example to set the maximum number of clients to 100000, while starting the server.

1. redis-server --maxclients 100000
-

Client Commands

Index	Command	Description
1	CLIENT LIST	Returns the list of clients connected to Redis server
2	CLIENT SETNAME	Assigns a name to the current connection
3	CLIENT GETNAME	Returns the name of the current connection as set by CLIENT SETNAME
4	CLIENT PAUSE	This is a connections control command able to suspend all the Redis client connections for a given amount of time (in milliseconds)
5	CLIENT KILL	This command closes a given client connection.

Next Topic [Redis Pipelining](#)

Redis Pipelining

Before knowing pipelining, first know the concept of Redis:

Redis is a TCP server which supports request/response protocol. In Redis, a request is completed in two steps:

- The client sends a query to the server usually in a blocking way for the server response.
 - The server processes the command and sends the response back to the client.
-

What is Pipelining

Pipelining facilitates a client to send multiple requests to the server without waiting for the replies at all, and finally reads the replies in a single step.

Example

Let's see an example of Redis pipelining. In this example, we will submit multiple commands to Redis once and Redis will provide the output of all commands in a single step.

Open Redis terminal and use the following command:

1. `(echo -en "PING\r\n SET sssit javatraining\r\n GET sssit\r\n INCR visitor\r\n INCR visitor\r\n INCR visitor\r\n"; sleep 10) |`
2. `nc localhost 6379`

```
sssit@sssit-Inspiron-15-3567:~$ (echo -en "PING\r\n SET sssit javatraining\r\n G
ET sssit\r\n INCR visitor\r\n INCR visitor\r\n INCR visitor\r\n"; sleep 10) | nc
localhost 6379
+PONG
+OK
$12
javatraining
:14
:15
:16
sssit@sssit-Inspiron-15-3567:~$ (echo -en "PING\r\n SET sssit javatraining\r\n G
ET sssit\r\n INCR visitor\r\n INCR visitor\r\n INCR visitor\r\n"; sleep 10) | nc
localhost 6379
+PONG
+OK
$12
javatraining
:17
:18
:19
sssit@sssit-Inspiron-15-3567:~$
```

Here:

- PING command is used to check Redis connection.
- A string is set named "sssit" having a value "javatraining".
- Got the key values and incremented the visitors numbers three times.

You can see that every time the value is incremented.

Advantage of Pipelining

The main advantage of Redis pipelining is speeding up the Redis performance. It drastically improves the protocol performance because of multiple commands simultaneous execution.

Pipelining vs Scripting

Redis Scripting is available in Redis version 2.6 or greater. A lot number of use cases for pipelining can be addressed more efficiently using scripts that perform a lot of the work needed at the server side.

The main advantage of scripting is that it can both read and write data with minimal latency. It makes operations like read, compute, write very fast.

On the other hand, pipelining doesn't use this scenerio. In pipelining, the client needs the reply of the read command before it can call the write command.

Next Topic [Redis Partitioning](#)

Redis Partitioning

Partitioning is used to split your Redis data into multiple Redis instances so that every instance contain only a subset of your keys.

It is generally used for large databases.

Types of Partitioning

There are two types of partitioning in redis:

- Range Partitioning
 - Hash Partitioning
-

Range Partitioning

Range partitioning is one of the simplest way to perform partitioning. It is done by mapping ranges of objects into specific Redis instances.

For example:

Suppose that you have 3000 users. So you can say that users from ID 0 to ID 1000 will go into instance R0, while users form ID 1001 to ID 2000 will go into instance R1, users form ID 2001 to ID 3000 will go into instance R2 and so forth.

Hash Partitioning

Hash partitioning is an alternative of Range partitioning. In Hash partitioning, a hash function is used to convert the key into a number and then the data is stored in different-different Redis instances.

Advantage of Redis Partitioning

- Partitioning facilitates you to use the collective memory of multiple computers. For example: In the case of larger databases you need a large amount of memory, so partitioning provides sum of memory from different computers. Without partitioning you can use only a limited amount of memory that a single computer can support.
- Partitioning is also used for scaling the computational power to multiple cores and multiple computers, and the network bandwidth to multiple computers and network adapters.

Disadvantage of Redis Partitioning

There are some disadvantages of partitioning because some features of Redis are obstructed with partitioning.

- Partitioning doesn't usually support the operations which have multiple keys. For example, you can't perform the intersection between two sets if they are stored in keys that are mapped to different Redis instances.
- Partitioning doesn't support the transactions which have multiple keys.
- The partitioning granularity is the key, so it is not possible to shard a dataset with a single huge key like a very big sorted set.
- When partitioning is used, data handling is more complex, for instance you have to handle multiple RDB / AOF files, and to make a backup of your data you need to aggregate the persistence files from multiple instances and hosts.
- Adding and removing capacity can be complex. For instance Redis Cluster supports mostly transparent rebalancing of data with the ability to add and remove nodes at runtime, but other systems like client side partitioning and proxies don't support this feature. However a technique called Pre-sharding helps in this regard.

Next Topic <#>

Redis Vs Memcached

Comparison Index	Memcached	Redis
What is	Memcached is in-memory key-value store, originally intended for caching purpose.	Redis is in-memory data store which can be used as database, cache and message broker.
Description	MemcachedD is easy, simply designed yet powerful. Its simple design promotes fast deployment, ease of exaggeration, and solves many problems related to large data caches. It has its inbuilt APIs which provide a very large hash table distributed across multiple machines & uses internal memory management to provide more efficiency. MemcachedD supports only String data type which are ideal for storing read-only data. Memcached is a volatile in-memory key-value origin. It is multi-threaded and used primarily for caching objects.	Redis is an open source in-memory data structure store which also can be used as database as well as caching system. It supports almost all types of data structures like strings, hashes, lists, sets, ordered sets, range queries, bitmaps, hyperloglogs, and geospatial indexes through geotools. Redis also can be used for real-time analytics and used as pub/sub.
Primary database model	Memcached follows Key-value store database model.	Redis also follows Key-value store database model.
Developed By	Memcached is developed by Danga Interactive.	Redis is developed by Salvatore Sanfilippo.
Initial release	Memcached was initially released in 2003.	Redis was initially released in 2009.
Current release	Memcached current version is 1.5.1, released in August 2017.	Redis current version is 4.0.1, released in September 2017.
License	Memcached is free and open source.	Redis is also free and open source.
Cloud-based	No	No
Implementation language	Memcached is implemented in C language.	Redis is also implemented in C language.

Server operating systems	FreeBSD Linux OS X Unix Windows	BSD Linux OS X Windows
Supported programming languages	.Net, C, C++, ColdFusion, Erlang, Java, Lisp, Lua, OCaml, Perl, PHP, Python, Ruby	C,C#,C++, Clojure, Crystal, Erlang, Fancy, Go, Haskell, JavaScript (Node.js), Lisp, Objective-C, OCaml info, P, Pure Data, Python, R, Rebol, Scala, Scheme, Smalltalk,
Server-side scripts	No	Lua
Triggers	No	No
Partitioning methods	None	Sharding
Replication methods	None	Master-slave replication
MapReduce	No	No
Foreign keys	No	No
Transaction concepts	No	Optimistic locking, atomic commands blocks and scri
Concurrency	Yes	Yes
Durability	No	Yes
User concepts	Yes	Simple password-based ac
Installation	Memcached is little bit complicated to install and run.	Installing Redis is so much dependencies required.
Memory Usage	MemcachedD is more memory efficient than Redis because it consumes comparatively less memory resources for metadata.	Redis is more memory effi use Redis hashes.

Persistence	Memcached doesn't use persistent data. While using Memcached, data might be lost with a restart and rebuilding cache is a costly process.	Redis can handle persistence by saving data to the disk at least every 90 seconds, offering optional RDB and AOF persistence meant to bootstrapping after a planned shutdown or an unexpected failure. While we tend to regard caches as volatile and transient, saving data to disk can be quite valuable in some scenarios.
Replication	Memcached does not support replication.	Redis supports master-slave replication.
Storage type	Memcached stores variables in its memory and retrieves any information directly from the server memory instead of hitting the database again.	Redis is like a database that stores data in memory. It executes (reads/writes) key/value pair from its database and returns the result set. That's why it is often used for real-time metrics and analytics.
Execution Speed and Performance	Memcached is very good to handle high traffic websites. It can read many information at a time and give you back at a great response time.	Redis can neither handle high read nor heavy writes.
Data Structure	Memcached uses only strings and integers in its data structure. So, everything you save can either be a string or an integer. It is complicated because with integers, the only data manipulation you can do is adding or subtracting them. If you need to save arrays or objects, you will have to serialize them first and then save them. To read them back, you will need to un-serialize.	Redis has stronger data structure and can handle not only strings integers but also binary-safe strings, lists of strings, sets of binary-safe strings, and sorted sets.
Key Length	Memcached's key length has a maximum of 250 bytes.	Redis' key-length has a maximum of 512 bytes.

Next Topic [Redis Vs MongoDB](#)

Redis Vs MongoDB

Comparison Index	Redis	MongoDB
Introduction	Redis is in-memory data structure store, used as database, cache and message broker.	MongoDB is one of the most database which follows the document structure.
Primary database model	Redis follows key-value store model.	MongoDB follows document s
Official Website	redis.io	www.mongodb.com
Technical Documentation	You can get technical documentation of Redis on redis.io/documentation	You can get technical docume on docs.mongodb.com/manu
Developed By	Redis is developed by Salvatore Sanfilippo.	MongoDB is developed by Mo
Initial Release	Redis is initially released in 2009.	MongoDB is also initially rele
Licence	Redis is subscription based and open-source.	MongoDB is free to use and c
Cloud based	No	No
Implementation Language	Redis is written and implemented in C language.	MongoDB is written and imple language.
Server operating systems	BSD, Linux, OS X, Windows	Linux, OS X, Solaris, Window
Data Scheme	schema-free	schema-free
Secondary Indexes	No	Yes
SQL	No	No
APIs and other access methods	Redis follows proprietary protocol.	MongoDB follows proprietary JSON.

Supported programming languages	C, C#, C++, Clojure, Crystal, D, Dart, Elixir, Erlang, Fancy, Go, Haskell, Haxe, Java, JavaScript (Node.js), Lisp, Lua, MatLab, Objective-C, OCaml, Perl, PHP, Prolog, Pure Data, Python, R, Rebol, Ruby, Rust, Scala, Scheme, Smalltalk, Tcl	Actionscript, C, C#, C++, Clojure, Dart, Delphi, Erlang, Go, Groovy, JavaScript, Lisp, Lua, MatLab, PowerShell, Prolog, Python, R, Smalltalk
Server-side scripts	Lua	JavaScript
Triggers	No	No
Partitioning methods	Redis uses Sharding for partition.	MongoDB also uses Sharding
Replication methods	Redis follows master-slave replication.	MongoDB also follows master-slave replication.
MapReduce	No	Yes
Consistency concepts	Eventual Consistency and Immediate Consistency	Eventual Consistency
Foreign keys	No	No
Transaction concepts	Optimistic locking, atomic execution of commands blocks and scripts.	No
Concurrency	Yes	Yes
MapReduce	No	Yes
Durability	Yes	Yes
In-memory capabilities	Yes	Yes
User concepts	Simple password-based access control.	Access rights for users and roles
Special	Redis is ranked as world's fastest database.	MongoDB is considered as the fastest NoSQL database.

Characteristics	It reduces application complexity, simplifies development, accelerates time to market and provides unprecedented flexibility to developers with its visionary data structures and modules.	database. It successfully helped companies to transform their industries with their data. The world's most sophisticated organizations, from cutting-edge startups to the largest companies, use MongoDB to build applications never before possible at a low cost.
Comparing Advantages	Redis is an in-memory database platform that provides support of a wide range of data structures such as strings, hashes, sets, lists, sorted sets, bitmaps, hyperloglogs, and geospatial indexes. Redis provides effortless scaling in a fully automated manner by overseeing all the operations of sharding, re-sharding, migration. It also includes persistence, instant automatic failure detection, backup and recovery, and in-memory replication across racks, zones, datacenters, regions, and cloud platforms.	MongoDB provides the best of both worlds: a NoSQL database as well as flexibility and performance required by today's applications. MongoDB is a database of giant scale that keeps the most valuable features of a relational database i.e. strong consistency, ACID, and a rich query language and secondary indexes. It facilitates developers to build applications faster than NoSQL.
Key Customers	Key customers of Redis are: Verizon, Vodafone, Atlassian, Trip Advisor, Jet.com, Nokia, Samsung, HTC, Docker, Staples, Intuit, Groupon, Shutterfly, KPMG, TD Bank, UnitedHealthcare, RingCentral, The Motley Fool, Bleacher Report, HipChat, Salesforce, Hotel Tonight, Cirruspath, Itslearning.com, Xignite, Chargify, Rumble Entertainment, Scopely, Havas Digital, Revmob, MSN, Bleacher Report, Mobli, TMZ, Klarna, Shopify etc.	Key customers of MongoDB are: AstraZeneca, BBVA, Bosch, CERN, Department of Veteran Affairs, Electronic Arts, Expedia, Facebook, Forbes, Foursquare, Genentech, Google, Sage, Salesforce, The Weather Channel, Ticketmaster, Under Armour, etc.
Market Metrics	Redis Labs consists of more than 60,000 customers globally and is consistently ranked as a leader in top analyst reports on NoSQL, in-memory and operational databases. Redis	20 million downloads (growing at 100,000 downloads per day). More than 100,000 developers are using MongoDB, including over one third of the Fortune 500. MongoDB is Named a leader in the Forrester

	is rated as no. 1 cloud database, no.1 database in Docker, no.1 NoSQL datastore, most popular NoSQL database in container.	NoSQL, Q3 2016. Highest pla database in DB Engines ranki
--	--	--

Next Topic [Redis vs Elasticsearch](#)

Redis vs Elasticsearch

Comparison Index	Redis	Elasticsearch
Introduction	Redis is in-memory data structure store, used as database, cache and message broker	Elasticsearch is a modern engine based on Apache L
Primary database model	Key-value store	Search Engine
DB-Engines Ranking	Score120.41 Overall Rank #9 Rank in Key-value stores #7	Score 120.00 Overall Rank Search engines #1
Website	redis.io	www.elastic.co/→products
Technical documentation	redis.io/→documentation	www.elastic.co/→guide/→i
Developed By	Salvatore Sanfilippo	Elastic
Initial release	2009	2010
Current release	4.0.2, September 2017	5.6.1, September 2017
License info	Open Source info	Open Source info
Cloud-based info	no	no
Implementation language	C	Java
Server operating systems	BSD Linux OS X Windows	All OS with a Java VM
Data scheme	schema-free	schema-free
Typing	partial	yes
XML support		no

Secondary indexes	no	yes
SQL	no	no
APIs and other access methods	proprietary protocol	Java API RESTful HTTP/JS
Supported programming languages	C C# C++ Clojure Crystal D Dart Elixir Erlang Fancy Go Haskell Haxe Java JavaScript (Node.js) Lisp Lua MatLab Objective-C OCaml Perl PHP Prolog Pure Data Python R Rebol Ruby Rust Scala Scheme Smalltalk Tcl	.Net Clojure Erlang Go Groovy JavaScript Lua Perl PHP Python
Server-side scripts	Lua	yes
Triggers	no	yes
Partitioning methods	Sharding	Sharding
Replication methods	Master-slave replication	yes
MapReduce	no	no
Consistency concepts	Eventual Consistency	Eventual Consistency
Foreign keys	no	no

Next Topic [Redis Vs Aerospike](#)

Redis Vs Aerospike

Comparison Index	Redis	Aerospike
Introduction	Redis is in-memory data structure store, used as database, cache and message broker.	Aerospike is a flash-optimized NoSQL Database.
Primary database model	Key-value store	Key-value store
Official Website	redis.io	www.aerospike.com
Technical documentation	redis.io/documentation	www.aerospike.com/docs
Developed By	Salvatore Sanfilippo	Aerospike
Initial release	2009	2012
License	Open Source	Open Source
Cloud-based	No	No
Implementation language	Redis is written and implemented in C language.	Aerospike is also written and implemented in C language.
Server operating systems	BSD, Linux, OS X, Windows	Linux
Data scheme	schema-free	schema-free
XML support	No	No
Secondary indexes	No	Yes
SQL	No	No
APIs and other	proprietary protocol	Proprietary protocol,JDBC

access methods		
Supported programming languages	C, C#, C++, Clojure, Crystal, D, Dart, Elixir, Erlang, Fancy, Go, Haskell, Haxe, Java, JavaScript (Node.js), Lisp, Lua, MatLab, Objective-C, OCaml info, Perl, PHP, Prolog, Pure Data, Python, R, Rebol, Ruby, Rust, Scala, Scheme, Smalltalk, Tcl	.Net, C, C#, C++, Erlang, G (Node.js), Lua, Perl, PHP, Py
Server-side scripts	Lua	user defined functions
Triggers	No	No
Partitioning methods	Sharding	Sharding
Replication methods	Master-slave replication	Selectable replication factor
MapReduce	No	Yes
Consistency concepts	Eventual Consistency	Eventual Consistency in cross configuration and Immediate local cluster configuration
Foreign keys	No	No
Transaction concepts	Optimistic locking, atomic execution of commands blocks and scripts	Atomic execution of operations
Concurrency	Yes	Yes
Durability	Yes	Yes
In-memory capabilities	Yes	Yes
User concepts	Simple password-based access control	Access rights for users and roles

Special Characteristics	Redis is ranked as world's fastest database. It reduces application complexity, simplifies development, accelerates time to market and provides unprecedented flexibility to developers with its visionary data structures and modules.	Aerospike is an enterprise-class database solution delivering performance at scale, superior availability at the lowest cost per generation NoSQL or relational database. Aerospike was mainly built with a parallelized architecture to support Big Data-driven applications that scale up and out.
Comparing Advantages	Redis is an in-memory database platform provides support of wide range of data structures such as strings, hashes, sets, lists, sorted sets, bitmaps, hyperloglogs, and geospatial indexes. Redis provides effortless scaling in a fully automated manner by overseeing all the operations of sharding, re-sharding, migration. It also includes persistence, instant automatic failure detection, backup and recovery, and in-memory replication across racks, zones, datacenters, regions, and cloud platforms.	Optimized for use with SSDs (solid state drives) Scales horizontally across nodes and shards Sharding & auto-healing Smart compression for mission-critical deployments Used by leading companies across many industries
Key Customers	Key customers of Redis are: Verizon, Vodafone, Atlassian, Trip Advisor, Jet.com, Nokia, Samsung, HTC, Docker, Staples, Intuit, Groupon, Shutterfly, KPMG, TD Bank, UnitedHealthcare, RingCentral, The Motley Fool, Bleacher Report, HipChat, Salesforce, Hotel Tonight, Cirruspath, Itslearning.com, Xignite, Chargify, Rumble Entertainment, Scopely, Havas Digital, Revmob, MSN, Bleacher Report, Mobli, TMZ, Klarna, Shopify etc.	Key customers of Aerospike are: Williams-Sonoma, Inmobi, Airtel (confidential), Financial Services Partners include Intel, Samsung Services, Cloudera, Dell and others
Market Metrics	Redis Labs consists of more than 60000 customers globally and is consistently ranked	Aerospike is a combination of performance and reliability. It delivers 10x

	as a leader in top analyst reports on NoSQL, in-memory and operational databases. Redis is rated as no. 1 cloud database, no.1 database in Docker, no.1 NoSQL datastore, most popular NoSQL database in container.	or 1/10th the cost compared
--	--	-----------------------------